

Databazy

June 4, 2026

1 Sémantika dotazov

1.1 Datalogový program ako teória

Datalogový program možno chápať ako axiomatickú teóriu. Teória je množina formúl, resp. axióm. V prípade Datalogu sú axiómami pravidlá tvaru

$$H \leftarrow B_1, \dots, B_n,$$

kde H je hlava pravidla a $B_1, \dots, B_n$ sú podmienky v tele pravidla.

Databáza sa zvyčajne delí na:

- **EDB** – extensional database, teda základné fakty uložené v databáze,
- **IDB** – intensional database, teda odvodené predikáty definované pravidlami.

Datalogový program spolu s EDB a prípadne dotazom tvorí logickú teóriu. Pravidlá programu možno chápať ako implikácie, pričom premenné v pravidlách sú univerzálne kvantifikované.

1.2 Modely teórie

Model teórie je interpretácia, v ktorej platia všetky axiomy danej teórie. Inými slovami, model je taká množina faktov, ktorá neporušuje pravidlá programu.

Minimálny model je model, ktorý neobsahuje žiadny vlastný neprázdny podmodel, ktorý by bol tiež modelom teórie. Intuitívne ide o model, ktorý obsahuje iba tie fakty, ktoré sú nevyhnutne odvodené z pravidiel a EDB.

Pre Datalog bez negácie platí:

$$\text{minimálny model} = \text{najmenší model} = \text{pevný bod.}$$

To znamená, že všetky fakty pravdivé v minimálnom modeli sú práve tie, ktoré vieme odvodiť iteratívnym aplikovaním pravidiel programu.

1.3 Sémantika pevného bodu

Pri Datalogu bez negácie definujeme bezprostredný dôsledkový operátor

$$T_P(I),$$

kde P je program a I je aktuálna interpretácia. Operátor T_P vráti všetky fakty, ktoré možno odvodiť jednou aplikáciou pravidiel programu z interpretácie I .

Výpočet začína od EDB:

$$I_0 = EDB.$$

Potom opakovane aplikujeme:

$$I_{i+1} = T_P(I_i).$$

Keď nastane stav

$$I_{i+1} = I_i,$$

dosiahli sme pevný bod. Tento pevný bod je výsledkom programu.

Pre Datalog bez negácie je tento proces monotónny, pretože nové odvodené fakty sa iba pridávajú a nikdy sa neodoberajú. Preto na konečnej databáze výpočet skončí po konečnom počte krokov.

1.3.1 Príklad

Majme pravidlá:

$$\begin{aligned} path(X, Y) &\leftarrow edge(X, Y). \\ path(X, Y) &\leftarrow edge(X, Z), path(Z, Y). \end{aligned}$$

Ak

$$EDB = \{edge(1, 2), edge(2, 3)\},$$

potom postupne odvodíme:

$$path(1, 2), \quad path(2, 3), \quad path(1, 3).$$

Pevný bod bude:

$$EDB \cup \{path(1, 2), path(2, 3), path(1, 3)\}.$$

1.4 Datalog s negáciou

Ak v pravidlách povolíme negované podciele, teda literály tvaru

$$\neg A,$$

situácia sa komplikuje. Na rozdiel od pozitívneho Datalogu už nemusí existovať jednoznačný minimálny model a naivná iterácia nemusí skončiť.

Typický problematický príklad je:

$$p \leftarrow \neg p.$$

Ak p neplatí, pravidlo hovorí, že p má platiť. Ak však p platí, potom podmienka $\neg p$ neplatí. Výpočet teda osciluje a nemá prirodzený pevný bod.

Datalog s negáciou môže mať:

- žiadny stabilný model,
- jeden stabilný model,
- viacero stabilných modelov,
- fakty, ktorých pravdivosť nemožno jednoznačne určiť.

1.5 Stratifikovaná negácia

Cieľom stratifikácie je zakázať cykly cez negáciu. Predikáty rozdelíme do vrstiev, tzv. **strát**. Každému IDB predikátu priradíme číslo vrstvy.

Ak má pravidlo tvar

$$p(\dots) \leftarrow q(\dots),$$

kde q sa v tele vyskytuje pozitívne, potom musí platiť:

$$\text{stratum}(p) \geq \text{stratum}(q).$$

Ak má pravidlo tvar

$$p(\dots) \leftarrow \neg q(\dots),$$

kde q sa v tele vyskytuje negovane, potom musí platiť:

$$\text{stratum}(p) > \text{stratum}(q).$$

Negovaný predikát teda musí byť vypočítaný v nižšej vrstve ako predikát, ktorý ho používa. Vďaka tomu možno program vyhodnocovať po vrstvách.

1.5.1 Intuícia

Najprv vypočítame všetky fakty v najnižšej vrstve. Keď sú hotové, môžeme ich použiť aj negovane vo vyšších vrstvách, pretože už vieme, čo v nižšej vrstve platí a čo neplatí.

Stratifikovaný program má jednoznačnú sémantiku a jeho výsledok zodpovedá pevnému bodu počítanému po jednotlivých stratách.

1.6 Lokálne stratifikovaná negácia

Lokálna stratifikácia je všeobecnejšia ako klasická stratifikácia. Rozdiel je v tom, že nestratifikujeme iba predikáty, ale konkrétne inštancované IDB atómy.

Pri lokálnej stratifikácii závisí stratifikácia nielen od programu, ale aj od konkrétnej EDB.

Postup:

1. Vytvoríme graf závislostí inštancovaných IDB atómov.
2. Vrcholy grafu sú konkrétne IDB atómy, napríklad $\text{win}(1)$, $\text{win}(2)$.
3. Hrana $p \rightarrow q$ existuje vtedy, ak atóm q vystupuje v tele pravidla s hlavou p .

4. Ak je q v tele negovaný, hrana sa označí ako negovaná.
5. Program je lokálne stratifikovaný, ak graf závislostí neobsahuje cyklus s negovanou hranou.

Pre lokálne stratifikované programy je zaručené:

$$\text{minimálny model} = \text{pevný bod.}$$

1.6.1 Príklad

Majme program:

$$\text{win}(X) \leftarrow \text{move}(X, Y), \neg \text{win}(Y).$$

Nech:

$$\text{EDB} = \{\text{move}(1, 2), \text{move}(1, 3), \text{move}(2, 3)\}.$$

Inštancované pravidlá sú:

$$\text{win}(1) \leftarrow \text{move}(1, 2), \neg \text{win}(2).$$

$$\text{win}(1) \leftarrow \text{move}(1, 3), \neg \text{win}(3).$$

$$\text{win}(2) \leftarrow \text{move}(2, 3), \neg \text{win}(3).$$

Atómy možno zaradiť do vrstiev:

$$\text{win}(3) \in S_1, \quad \text{win}(2) \in S_2, \quad \text{win}(1) \in S_3.$$

Preto je program pre danú EDB lokálne stratifikovaný. Výsledný model je:

$$\text{EDB} \cup \{\text{win}(1), \text{win}(2)\}.$$

1.7 Gelfondova–Lifschitzova transformácia

Gelfondova–Lifschitzova transformácia, skrátene GL transformácia alebo GLT, prevádza Datalogový program s negáciou na pozitívny Datalogový program bez negovaných IDB podcieľov. Transformácia sa robí vzhľadom na danú interpretáciu M .

Nech P je program a M je interpretácia. GL transformácia sa vykoná takto:

1. Inštancujeme pravidlá programu všetkými možnými spôsobmi.
2. Odstránime každé pravidlo, ktoré obsahuje negovaný literál $\neg A$, kde $A \in M$.
3. Zo zvyšných pravidiel odstránime všetky negované literály $\neg A$, kde $A \notin M$.
4. Zo vzniknutého pozitívneho programu vypočítame minimálny model.

Výsledný minimálny model označíme:

$$GLT(M).$$

Formálne, ak máme pravidlo:

$$H \leftarrow P_1, \dots, P_n, \neg N_1, \dots, \neg N_k,$$

potom v transformovanom programe ostane pravidlo:

$$H \leftarrow P_1, \dots, P_n$$

práve vtedy, keď:

$$N_1, \dots, N_k \notin M.$$

Ak aspoň jeden negovaný atóm N_i patrí do M , pravidlo sa celé odstráni.

1.7.1 Príklad GL transformácie

Majme program:

$$win(X) \leftarrow move(X, Y), \neg win(Y).$$

Nech:

$$EDB = \{move(1, 2), move(1, 3), move(2, 3)\}$$

a interpretácia:

$$M = EDB \cup \{win(1), win(2)\}.$$

Inštancované pravidlá sú:

$$win(1) \leftarrow move(1, 2), \neg win(2).$$

$$win(1) \leftarrow move(1, 3), \neg win(3).$$

$$win(2) \leftarrow move(2, 3), \neg win(3).$$

Prvé pravidlo odstránime, pretože $win(2) \in M$.

V druhom a treťom pravidle odstránime negované podmienky $\neg win(3)$, pretože $win(3) \notin M$.

Dostaneme:

$$win(1).$$

$$win(2).$$

Teda:

$$GLT(M) = EDB \cup \{win(1), win(2)\}.$$

1.8 Stabilné modely

Interpretácia M je **stabilný model** programu P , ak platí:

$$M = GLT(M).$$

Inými slovami, stabilný model je taká interpretácia, ktorá sa po Gelfondovej–Lifschitzovej transformácii nezmení.

Stabilné modely teda vyjadrujú samoopodstatnené množiny faktov. Interpretácia M je stabilná, ak predpoklady o negovaných atónoch, ktoré sme použili pri transformácii, vedú späť k tej istej interpretácii.

1.8.1 Problém viacerých stabilných modelov

Program s negáciou môže mať viac stabilných modelov. Napríklad pre program:

$$win(X) \leftarrow move(X, Y), \neg win(Y)$$

a EDB:

$$\{move(1, 2), move(2, 1), move(2, 3), move(3, 4), move(4, 5), move(5, 6)\}$$

môžu existovať dva stabilné modely:

$$EDB \cup \{win(1), win(3), win(5)\},$$

$$EDB \cup \{win(2), win(3), win(5)\}.$$

Pre dotaz

$$? - win(X)$$

by sme teda mohli dostať rôzne odpovede:

$$\{1, 3, 5\}$$

alebo

$$\{2, 3, 5\}.$$

To je problém, pretože databázové dotazy by mali mať jednoznačný výsledok. Preto sa často používa aj well-founded sémantika.

1.9 Well-founded model

Well-founded model je jedinečný trojhodnotový model. Každý IDB atóm má jednu z troch hodnôt:

$$true, \quad false, \quad unknown.$$

Teda atóm môže byť:

- pravdivý,

- nepravdivý,
- neurčený.

Well-founded model sa dá vypočítať pomocou opakovanej GL transformácie, tzv. alternating fixpoint metódou. Proces začína prázdnu parciálnou interpretáciou. V iteráciách sa postupne spresňuje, ktoré atómy sú určite pravdivé a ktoré sú určite nepravdivé. Pre konečný počet IDB atómov tento proces skonverguje po konečnom počte krokov.

Dôležité vlastnosti well-founded modelu:

- Ak je IDB atóm pravdivý vo well-founded modeli, potom je pravdivý vo všetkých stabilných modeloch.
- Ak je IDB atóm nepravdivý vo well-founded modeli, potom je nepravdivý vo všetkých stabilných modeloch.
- Ak well-founded model neobsahuje žiadne neznáme IDB atómy, potom existuje práve jeden stabilný model a ten sa rovná well-founded modelu.

Pri databázových dotazoch sa zvyčajne do výsledku vracajú iba pravdivé fakty. Rozdiel medzi hodnotami *false* a *unknown* sa pri odpovedaní na dotazy často neprejaví priamo, pretože ani jedna z nich sa vo výsledku nevracia.

1.10 Vzťahy medzi sémantikami

Pre rôzne triedy programov platí:

Datalog bez negácie $\subseteq$ stratifikované programy $\subseteq$ lokálne stratifikované programy.

Pre Datalog bez negácie platí:

minimálny model = pevný bod.

Pre stratifikované a lokálne stratifikované programy je tiež zaručený jednoznačný výsledok získaný výpočtom po vrstvách.

Pre všeobecné programy s negáciou používame všeobecnejšie sémantiky:

- stabilné modely,
- well-founded model,
- prípadne inflačnú sémantiku.

Stabilná sémantika môže dať viacero modelov alebo žiadny model. Well-founded sémantika dáva vždy jeden model, ale môže ponechať niektoré atómy ako *unknown*.

1.11 Krátke zhrnutie na skúšku

Datalogový program možno chápať ako logickú teóriu, ktorej modely sú interpretácie spĺňajúce všetky pravidlá. Pri Datalogu bez negácie existuje jedinečný najmenší model, ktorý sa rovná pevnému bodu získanému iteratívnym aplikovaním pravidiel. Po zavedení negácie vznikajú problémy: nemusí existovať pevný bod, môže existovať viacero minimálnych alebo stabilných modelov.

Stratifikovaná negácia rieši problém tak, že zakáže cykly cez negáciu medzi predikátmi. Lokálna stratifikácia je jemnejšia, pretože sa aplikuje na konkrétne inšancované atómy a závisí aj od EDB. Pre lokálne stratifikované programy je zaručené, že minimálny model sa rovná pevnému bodu.

Pre všeobecné programy s negáciou sa používa Gelfondova–Lifschitzova transformácia. Tá vzhľadom na interpretáciu M odstráni pravidlá odporujúce M , odstráni zvyšné negácie a vypočíta minimálny model pozitívneho programu. Ak platí $M = GLT(M)$, potom M je stabilný model.

Well-founded model je jedinečný trojhodnotový model, v ktorom sú atómy pravdivé, nepravdivé alebo neznáme. Je konzervatívny: čo označí ako pravdivé, platí vo všetkých stabilných modeloch; čo označí ako nepravdivé, je nepravdivé vo všetkých stabilných modeloch.

2 Optimalizácia rekurzívnych dotazov

2.1 Datalog, Prolog a rekurzívne dotazy

Datalog a Prolog používajú podobnú logickú syntax založenú na pravidlách, ale líšia sa spôsobom výpočtu.

Datalog sa typicky vyhodnocuje metódou **bottom-up**. To znamená, že začíname od faktov v EDB a opakovane aplikujeme pravidlá, kým nedosiahneme pevný bod.

Prolog sa vyhodnocuje metódou **top-down**. Výpočet začína dotazom a systém sa pokúša dokázať, že dotaz platí. Používa pritom SLD rezolúciu, unifikáciu a spätné prehľadávanie.

Pri nerekurzívnych dotazoch je rozdiel najmä výkonnostný. Pri rekurzívnych dotazoch môže byť rozdiel zásadný, pretože bottom-up výpočet môže odvodiť veľa faktov, ktoré nie sú pre konkrétny dotaz potrebné.

2.1.1 Príklad: tranzitívny uzáver

Majme pravidlá:

$$\begin{aligned} p(X, Y) &\leftarrow e(X, Y). \\ p(X, Y) &\leftarrow e(X, Z), p(Z, Y). \end{aligned}$$

Dotaz:

$$? - p(a, b).$$

Bottom-up výpočet najprv vypočíta celú reláciu p , teda všetky cesty v grafe. Až potom vyberie, či existuje cesta z a do b .

Top-down výpočet začne priamo dotazom $p(a, b)$ a hľadá iba dôkaz tejto konkrétnej skutočnosti.

2.2 Datalogové programy s funkčnými symbolmi

V klasickom Datalogu sa zvyčajne nepovoľujú funkčné symboly s aritou väčšou ako nula. Dôvodom je, že bez funkčných symbolov je Herbrandov univerzum konečné, a preto je konečný aj počet možných odvodených faktov.

Ak povolíme funkčné symboly, môžu vznikať zložitejšie termy:

$$f(a), \quad f(f(a)), \quad g(a, f(b)), \dots$$

Funkčný symbol alebo funktor vytvára nový term z jednoduchších termov. Konštanty možno chápať ako funkčné symboly arity 0.

Napríklad:

$$\text{role}(\text{menelaus}, \text{iliad}, \text{king}(\text{sparta}, \text{menelean})).$$

Tu je:

$$\text{king}(\text{sparta}, \text{menelean})$$

zložený term vytvorený funktorom $\text{king}/2$.

Ak existuje aspoň jeden funkčný symbol arity väčšej ako nula, Herbrandov univerzum môže byť nekonečné:

$$a, f(a), f(f(a)), f(f(f(a))), \dots$$

Preto rekurzívny Datalog s funkčnými symbolmi nemusí terminovať.

2.2.1 Príklad neterminácie

Majme program:

$$p(a).$$

$$p(f(X)) \leftarrow p(X).$$

Bottom-up výpočet odvodí:

$$p(a),$$

$$p(f(a)),$$

$$p(f(f(a))),$$

$$p(f(f(f(a)))) , \dots$$

Teda výpočet generuje nekonečne veľa faktov a nedosiahne konečný pevný bod.

2.3 Unifikácia termov

Unifikácia je proces hľadania substitúcie premenných, ktorá spôsobí, že dva termy budú syntakticky rovnaké.

Substitúcia je zobrazenie premenných na termy, napríklad:

$$\sigma = \{X \mapsto a, Y \mapsto f(a)\}.$$

Ak pre dva termy t_1 a t_2 existuje substitúcia σ taká, že:

$$t_1\sigma = t_2\sigma,$$

hovoríme, že termy sú unifikovateľné.

Najdôležitejšia je **najvšeobecnejšia unifikácia**, teda **MGU** – most general unifier.

2.3.1 Príklady unifikácie

$$p(X, b) \quad \text{a} \quad p(a, Y)$$

sa unifikujú substitúciou:

$$\sigma = \{X \mapsto a, Y \mapsto b\}.$$

$$f(X, a) \quad \text{a} \quad f(b, Y)$$

sa unifikujú substitúciou:

$$\sigma = \{X \mapsto b, Y \mapsto a\}.$$

$$f(X) \quad \text{a} \quad g(X)$$

sa neunifikujú, pretože majú rôzne hlavné funktory f a g .

$$f(X, X) \quad \text{a} \quad f(a, b)$$

sa neunifikujú, pretože by muselo platiť:

$$X = a \quad \text{a zároveň} \quad X = b.$$

2.4 Occurs-check

Pri korektnej unifikácii sa kontroluje, či sa premenná nevyskytuje v terme, ktorým má byť nahradená. Táto kontrola sa nazýva **occurs-check**.

Napríklad:

$$X = f(X)$$

by nemalo byť povolené, pretože by viedlo k nekonečnému termu:

$$X = f(f(f(\dots))).$$

Mnohé implementácie Prologu z efektívnostných dôvodov occurs-check štandardne nepoužívajú. Preto môžu prijať aj niektoré unifikácie, ktoré nie sú korektné v klasickej logike.

Korektná unifikácia by mala dať:

$$\text{unify_with_occurs_check}(X, f(X)) = \text{false}.$$

Ale:

$$\text{unify_with_occurs_check}(X, f(a)) = \text{true},$$

so substitúciou:

$$X \mapsto f(a).$$

2.5 Naivná evaluácia rekurzívnych Datalog programov

Pri bottom-up vyhodnocovaní definujeme operátor okamžitých dôsledkov:

$$T_P(I),$$

kde P je program a I je aktuálna množina známych faktov.

Výpočet začína:

$$I_0 = EDB.$$

Potom opakovane počítame:

$$I_{i+1} = I_i \cup T_P(I_i).$$

Keď:

$$I_{i+1} = I_i,$$

dosiahli sme pevný bod.

2.5.1 Naivná evaluácia

Naivná evaluácia v každom kroku znova aplikuje všetky pravidlá na všetky doteraz odvodené fakty.

Nevýhoda je, že sa opakovane prepočítavajú aj odvodenia, ktoré už boli vykonané v predchádzajúcich iteráciách.

2.5.2 Príklad

Majme:

$$\text{path}(X, Y) \leftarrow \text{edge}(X, Y).$$

$$\text{path}(X, Y) \leftarrow \text{edge}(X, Z), \text{path}(Z, Y).$$

Ak:

$$EDB = \{\text{edge}(a, b), \text{edge}(b, c), \text{edge}(c, d)\},$$

potom:

$$I_0 = EDB.$$

V prvej iterácii odvodíme:

$$path(a, b), path(b, c), path(c, d).$$

V ďalšej iterácii:

$$path(a, c), path(b, d).$$

V ďalšej:

$$path(a, d).$$

Potom už nič nové nepribudne a dosiahneme pevný bod.

2.6 Seminaivná evaluácia

Seminaivná evaluácia optimalizuje naivnú evaluáciu tým, že v každej iterácii používa iba **nové fakty**, ktoré pribudli v predchádzajúcej iterácii.

Označme:

$$\Delta I_i = I_i - I_{i-1}.$$

Namiesto toho, aby sme v každom kroku používali celé I_i , používame ΔI_i . Cieľom je odvodiť iba tie fakty, ktoré skutočne môžu byť nové.

2.6.1 Intuícia

Ak sa v predchádzajúcom kroku neobjavil nový fakt, nemôže jeho opätovné použitie vyprodukovať nič nové. Preto sa zameriavame iba na novo odvodené fakty.

2.6.2 Príklad pre tranzitívny uzáver

Pravidlo:

$$path(X, Y) \leftarrow edge(X, Z), path(Z, Y)$$

prepíšeme seminaivne tak, že v každej iterácii použijeme len novo odvodené cesty:

$$\Delta path_{i+1}(X, Y) = edge(X, Z) \bowtie \Delta path_i(Z, Y) - path_i(X, Y).$$

Potom:

$$path_{i+1} = path_i \cup \Delta path_{i+1}.$$

Výhoda je, že nepočítame stále tie isté joiny nad celou reláciou $path$.

2.7 Prolog

Prolog je logický programovací jazyk založený na predikátovej logike prvého rádu. Program pozostáva z faktov a pravidiel.

Fakty:

$male(achilles).$
 $female(helen).$
 $character(helen, iliad).$

Pravidlo má tvar:

$$HEAD : -GOAL_1, GOAL_2, \dots, GOAL_n.$$

Pravá strana pravidla je konjunkcia podcieľov. Dotazy sú syntakticky podobné telám pravidiel:

$$? - character(X, iliad), female(X).$$

2.8 SLD rezolúcia

Prolog používa na výpočet dotazov **SLD rezolúciu**. Ide o top-down dôkazovú procedúru.

SLD znamená:

- **S** – selection rule,
- **L** – linear resolution,
- **D** – definite clauses.

Základná myšlienka je, že Prolog sa pokúša dokázať cieľ tak, že hľadá pravidlo, ktorého hlava sa dá unifikovať s aktuálnym cieľom.

2.8.1 Algoritmus SLD rezolúcie

Majme aktuálny resolvent:

$$G_1, G_2, \dots, G_m.$$

Postup:

1. Vyberie sa cieľ G_i z resolventu.
2. Nájde sa pravidlo:

$$H : -B_1, \dots, B_n$$

také, že G_i a H sa unifikujú s MGU θ .

3. Cieľ G_i sa nahradí telom pravidla:

$$B_1, \dots, B_n.$$

4. Substitúcia θ sa aplikuje na celý resolvent a na pôvodný dotaz.

5. Ak je resolvent prázdny, dôkaz uspel.
6. Ak neexistuje použiteľné pravidlo, vetva výpočtu zlyhá.

Prolog štandardne vyberá ciele aj pravidlá deterministicky:

- podciele spracúva zľava doprava,
- pravidlá skúša v poradí, v akom sú uvedené v programe,
- pri zlyhaní používa spätné prehľadávanie.

2.8.2 Príklad SLD rezolúcie

Program:

$$\begin{aligned} mortal(X) &: \neg man(X). \\ man(socrates). \end{aligned}$$

Dotaz:

$$? \neg mortal(socrates).$$

Postup:

$$mortal(socrates)$$

sa unifikuje s hlavou:

$$mortal(X)$$

substitúciou:

$$\{X \mapsto socrates\}.$$

Cieľ sa nahradí telom pravidla:

$$man(socrates).$$

Fakt:

$$man(socrates)$$

existuje, teda dôkaz uspeje.

2.9 Rule-Goal Tree

Rule-Goal Tree, skrátene RGT, je stromová reprezentácia top-down výpočtu.

Koreňom stromu je počiatočný cieľ:

$$G_0.$$

Potom:

1. Pre cieľový uzol sa nájdu všetky pravidlá, ktorých hlavy sa dajú unifikovať s cieľom.
2. Každé také pravidlo vytvorí pravidlový uzol.

3. Premenné pravidla sa pred použitím premenujú na čerstvé premenné, aby nekolidovali s premennými aktuálneho cieľa.
4. Deti pravidlového uzla sú podciele z tela pravidla.
5. Každý podcieľ sa spracuje rekurzívne.

2.9.1 Príklad RGT

Program:

$$\begin{aligned} anc(X, Y) &\leftarrow par(X, Y). \\ anc(X, Y) &\leftarrow par(X, Z), anc(Z, Y). \end{aligned}$$

Dotaz:

$$? - anc(j, A).$$

Koreň:

$$anc(j, A).$$

Použijú sa dve pravidlá:

$$\begin{aligned} anc(j, A) &\leftarrow par(j, A). \\ anc(j, A) &\leftarrow par(j, Z_1), anc(Z_1, A). \end{aligned}$$

Druhé pravidlo vytvára rekurzívny podcieľ:

$$anc(Z_1, A),$$

ktorý sa znovu rozvíja rovnakými pravidlami.

Pre rekurzívne programy môže byť RGT nekonečný.

2.10 Rule-Goal Graph

Rule-Goal Graph, skrátene RGG, je grafová verzia RGT. RGG je zovšeobecnením RGT.

Na rozdiel od RGT je RGG konečný, ale môže obsahovať cykly. Cykly vznikajú, keď sa počas top-down výpočtu opakovane vyhodnocuje ten istý predikát alebo tá istá pozícia v pravidle s rovnakým vzorom väzieb.

RGG obsahuje:

- cieľové uzly, ktoré reprezentujú predikáty,
- uzly pozícií pravidiel,
- hrany medzi cieľmi a pravidlami,
- informáciu o tom, ktoré argumenty sú viazané.

Uzol s EDB predikátom nemá odchádzajúce hrany. Uzol s IDB predikátom má hrany do pravidiel, ktorých hlavy sa dajú s týmto cieľom unifikovať.

RGG je vhodný na optimalizáciu, pretože umožňuje zachytiť opakujúce sa volania a väzby medzi premennými bez vytvárania nekonečného stromu.

2.11 Vázby medzi premennými a binding patterns

Pri top-down výpočte je veľmi dôležité, ktoré argumenty cieľa sú už známe, teda **viazané**, a ktoré sú ešte neznáme, teda **voľné**.

Binding pattern, nazývaný aj adornment, označuje pri každom argumente, či je:

b bound,

alebo:

f free.

Napríklad pre predikát:

$p(X, Y)$

môžeme mať tieto vzory:

$p^{bb}, p^{bf}, p^{fb}, p^{ff}.$

Dotaz:

$? - p(a, Y)$

má vzor:

$p^{bf},$

pretože prvý argument je viazaný a druhý je voľný.

Dotaz:

$? - p(X, Y)$

má vzor:

$p^{ff}.$

Dotaz:

$? - p(a, b)$

má vzor:

$p^{bb}.$

2.12 Magic predicates a supplementary relations

Pri top-down optimalizácii sa väzby prenášajú pomocou tzv. **magic predicates**. Magic predikát reprezentuje hodnoty, ktoré sú už známe z predchádzajúceho výpočtu.

Pre cieľový uzol sa vytvorí väzbová relácia:

$M.$

Viazané premenné cieľa sú atribútmi tejto magic relácie.

Okrem toho sa v pravidlových uzloch používajú **supplementary relations**:

$S_0, S_1, \dots, S_n.$

Tieto relácie uchovávajú informácie prenášané zľava doprava cez podciele pravidla. Ide o tzv. **sideway-passed information**.

2.12.1 Príklad prenosu väzieb

Majme pravidlo:

$$p(X, Y) \leftarrow q(X, Z), r(Z, Y).$$

Dotaz:

$$? - p(a, Y).$$

Na začiatku vieme:

$$X = a.$$

Teda máme vzor:

$$p^{bf}.$$

Táto väzba sa prenese do prvého podcieľa:

$$q(a, Z).$$

Po vyhodnotení $q(a, Z)$ získame možné hodnoty Z . Tie sa potom prenese do druhého podcieľa:

$$r(Z, Y).$$

Tým sa znižuje množstvo dát, nad ktorými sa vykonáva join.

2.13 Optimalizácia rekurzívnych dotazov

Optimalizácia rekurzívnych dotazov sa snaží obmedziť výpočet iba na fakty, ktoré môžu prispieť k odpovedi na konkrétny dotaz.

Hlavné techniky:

- seminaivná evaluácia,
- využívanie väzieb z dotazu,
- top-down výpočet pomocou SLD rezolúcie,
- rule-goal-tree a rule-goal-graph,
- magic predicates,
- supplementary relations,
- transformácie podobné Magic Sets.

2.13.1 Bottom-up verus top-down

Bottom-up výpočet je vhodný, keď chceme vypočítať celú odvodenú reláciu. Nevýhodou je, že pre konkrétny dotaz môže vypočítať veľa nepotrebných faktov.

Top-down výpočet je vhodný, keď dotaz obsahuje viazané argumenty. Vtedy vie využiť tieto väzby a prehľadávať iba relevantnú časť priestoru.

2.14 Magická transformácia

Magická transformácia je optimalizačná technika pre rekurzívne Datalogové dotazy. Jej cieľom je spojiť výhody top-down a bottom-up výpočtu.

Top-down výpočet začína konkrétnym dotazom a prirodzene využíva väzby premenných z dotazu. Bottom-up výpočet je vhodný na relačné vyhodnocovanie, ale bez optimalizácie môže odvodiť aj veľa faktov, ktoré nie sú pre daný dotaz potrebné.

Magická transformácia pre daný dotaz transformuje pôvodný program P na nový program P^m , nazývaný **magický program**, tak, aby platilo:

- P^m vypočíta rovnaký výsledok ako pôvodný program P pre daný dotaz,
- seminaívne bottom-up vyhodnocovanie programu P^m počíta iba tie fakty, ktoré by boli navštívené top-down výpočtom pôvodného programu,
- ak je pôvodný program bezpečný, bezpečný je aj magický program.

Základnou myšlienkou je pridať do pravidiel pomocné predikáty, ktoré fungujú ako filtre. Tieto filtre hovoria, ktoré hodnoty sú relevantné vzhľadom na konkrétny dotaz.

2.14.1 Magic predikáty

Pre každý ozdobený IDB predikát sa vytvorí nový pomocný predikát, nazývaný **magic predicate**. Ozdobený predikát znamená, že pri jeho argumentoch vieme, ktoré sú viazané a ktoré voľné.

Používame označenie:

$$b = \text{bound}, \quad f = \text{free}.$$

Napríklad dotaz:

$$? - p(a, Y)$$

má vzor väzieb:

$$p^{bf},$$

pretože prvý argument je viazaný a druhý je voľný.

K tomuto ozdobenému predikátu sa vytvorí magic predikát:

$$m_p_bf.$$

Ak je prvý argument predikátu p viazaný, magic predikát obsahuje práve hodnoty tohto viazaného argumentu, ktoré sú relevantné pre výpočet.

2.14.2 Inicializácia magic predikátu

Pre dotaz:

$$? - q(B_1, \dots, B_m, F_1, \dots, F_n)$$

kde $B_1, \dots, B_m$ sú viazané argumenty, sa vytvorí počiatočný magic fakt:

$$m_q(B_1, \dots, B_m).$$

Tento fakt sa nazýva **seed**. Určuje počiatočné hodnoty, od ktorých sa má začať bottom-up výpočet magického programu.

2.14.3 Príklad

Majme program:

$$sg(X, X) \leftarrow person(X).$$

$$sg(X, Y) \leftarrow par(X, XP), sg(XP, YP), par(Y, YP).$$

Dotaz:

$$? - sg(j, Y).$$

Dotaz má vzor:

$$sg^{bf},$$

pretože prvý argument je viazaný na konštantu j a druhý argument je voľný.

Vytvoríme počiatočný magic fakt:

$$m_sg_bf(j).$$

Intuícia je taká, že nás zaujímajú iba hodnoty predikátu $sg(X, Y)$, ktoré sú dosiahnuteľné z počiatočnej hodnoty $X = j$. Nechceme počítat celú reláciu sg , ale iba jej relevantnú časť.

2.15 Supplementary predikáty

Okrem magic predikátov sa pri magickej transformácii používajú aj **supplementary predicates**. Tie reprezentujú medzivýsledky pri spracovaní tela pravidla zľava doprava.

Majme pravidlo:

$$q(\dots) \leftarrow G_1, G_2, \dots, G_l.$$

Pri spracovaní pravidla vznikajú pomocné predikáty:

$$sup_{i,0}, sup_{i,1}, \dots, sup_{i,l-1}.$$

Predikát $sup_{i,j}$ obsahuje hodnoty premenných, ktoré sú známe po spracovaní prvých j podcieľov pravidla. Ide teda o formalizáciu prenášania väzieb cez telo pravidla.

2.15.1 Intuícia

Ak máme pravidlo:

$$p(X, Y) \leftarrow q(X, Z), r(Z, Y),$$

a dotaz:

$$? - p(a, Y),$$

potom vieme, že:

$$X = a.$$

Táto informácia sa najskôr preniesie do podcieľa:

$$q(a, Z).$$

Po jeho vyhodnotení poznáme možné hodnoty Z . Tie sa potom preniesú do druhého podcieľa:

$$r(Z, Y).$$

Supplementary predikáty teda opisujú postupné šírenie informácií zľava doprava.

2.16 Preklad RGG do magického programu

Magická transformácia sa dá formulovať ako preklad z **Rule-Goal Graphu** do magického programu. Každá hrana RGG sa preloží na jedno pravidlo magického programu.

Rozlišuje sa päť tried hrán:

1. hrana z dotazu do prvého IDB cieľa,
2. hrana z iného uzla do IDB cieľa,
3. hrana z IDB cieľa do prvého uzla pravidla,
4. hrana z jedného uzla pravidla do nasledujúceho uzla pravidla,
5. hrana z uzla pravidla do posledného uzla pravidla, kde sa odvodzuje hlava pravidla.

2.16.1 Trieda 1: dotaz do IDB cieľa

Dotaz inicializuje magic predikát.

Ak máme:

$$? - q(B_1, \dots, B_m, F_1, \dots, F_n),$$

potom vytvoríme:

$$m_q(B_1, \dots, B_m).$$

2.16.2 Trieda 2: rekurzívne volanie IDB cieľa

Ak sa v tele pravidla nachádza IDB cieľ:

$$p(B_1, \dots, B_m, F_1, \dots, F_n),$$

potom sa preň vytvorí alebo aktualizuje magic predikát:

$$m_p(B_1, \dots, B_m) \leftarrow sup_{i,j-1}(\dots).$$

Tým sa do rekurzívneho volania prenášajú iba relevantné viazané hodnoty.

2.16.3 Trieda 3: vstup do pravidla

Magic predikát inicializuje supplementary predikát pravidla:

$$sup_{i,0}(B_1, \dots, B_m) \leftarrow m_q(B_1, \dots, B_m).$$

To znamená, že pravidlo sa použije iba pre tie hodnoty hlavy, ktoré sú povolené magic predikátom.

2.16.4 Trieda 4: prechod medzi podcieľmi pravidla

Prechod medzi podcieľmi pravidla sa preloží ako:

$$sup_{i,j+1}(\dots) \leftarrow sup_{i,j}(\dots), G_j.$$

V relačnej algebre ide o join:

$$sup_{i,j+1} := sup_{i,j} \bowtie G_j.$$

2.16.5 Trieda 5: odvodenie hlavy pravidla

Na konci pravidla sa z posledného supplementary predikátu a posledného podcieľa odvodí pôvodný IDB predikát:

$$q(B_1, \dots, B_m, F_1, \dots, F_n) \leftarrow sup_{i,l-1}(\dots), G_l.$$

2.17 Príklad magickej transformácie

Majme program:

$$r_1 : \quad sg(X, X) \leftarrow person(X).$$

$$r_2 : \quad sg(X, Y) \leftarrow par(X, XP), sg(XP, YP), par(Y, YP).$$

Dotaz:

$$? - sg(j, Y).$$

Ozdobený cieľ je:

$$sg^{bf}.$$

Magický program je:

$$m_sg_bf(j).$$

$$m_sg_bf(XP) \leftarrow sup2.1_bfbb(X, XP).$$

$$sup1.0_b(X) \leftarrow m_sg_bf(X).$$

$$sup2.0_bf_ff(X) \leftarrow m_sg_bf(X).$$

$$sup2.1_bfbb(X, XP) \leftarrow sup2.0_bf_ff(X), par(X, XP).$$

$$sup2.2_bfbb(X, XP, YP) \leftarrow sup2.1_bfbb(X, XP), sg(XP, YP).$$

$$sg(X, X) \leftarrow sup1.0_b(X), person(X).$$

$$sg(X, Y) \leftarrow sup2.2_bfbb(X, XP, YP), par(Y, YP).$$

Po zjednodušení, keď nahradíme supplementary predikáty ich definíciami, dostaneme:

$$m_sg_bf(j).$$

$$m_sg_bf(XP) \leftarrow m_sg_bf(X), par(X, XP).$$

$$sg(X, X) \leftarrow m_sg_bf(X), person(X).$$

$$sg(X, Y) \leftarrow m_sg_bf(X), par(X, XP), sg(XP, YP), par(Y, YP).$$

Dotaz ostáva:

$$? - sg(j, Y).$$

2.17.1 Význam príkladu

Pôvodný bottom-up výpočet by počítal celú reláciu sg . Magický program však najprv vypočíta iba tie hodnoty X , ktoré sú dosiahnuteľné z j cez reláciu par . Predikát:

$$m_sg_bf(X)$$

teda slúži ako filter. Pravidlá pre sg sa aplikujú iba pre také X , ktoré sú relevantné pre dotaz.

2.18 Zovšeobecnená magická transformácia

Zovšeobecnená magická transformácia rozširuje základnú magickú transformáciu tak, aby bola použiteľná aj v zložitejších situáciách.

Základná magická transformácia predpokladá relatívne jednoduchý prenos väzieb a pevné poradie spracovania podcieľov. V zložitejších programoch však môže existovať viac možných vzorov väzieb a viac možných poradí vyhodnocovania podcieľov.

Zovšeobecnená magická transformácia preto pracuje s:

- ozdobenými predikátmi,
- rôznymi binding patterns,
- Rule-Goal Graphom,
- magic predikátmi pre jednotlivé ozdobené IDB predikáty,
- supplementary predikátmi pre uzly pravidiel,
- bezpečnými a realizovateľnými poradiami podcieľov.

2.18.1 Binding patterns

Pre každý predikát sa môžu vytvoriť viaceré ozdobené verzie podľa toho, ktoré argumenty sú viazané.

Napríklad pre binárny predikát $p(X, Y)$ môžu existovať:

$$p^{bb}, \quad p^{bf}, \quad p^{fb}, \quad p^{ff}.$$

Každá verzia môže mať vlastný magic predikát:

$$m_p_bb, \quad m_p_bf, \quad m_p_fb, \quad m_p_ff.$$

V praxi sa však vytvárajú iba tie verzie, ktoré sú potrebné pre daný dotaz a ktoré sa môžu objaviť v Rule-Goal Graphe.

2.18.2 Realisable RGG

Zovšeobecnená magická transformácia začína od realizovateľného Rule-Goal Graphu. RGG opisuje, ktoré volania predikátov môžu vzniknúť pri top-down výpočte a s akými väzbami.

Pre každý ozdobený IDB predikát v RGG sa vytvorí magic predikát. Pre každý pravidlový uzol sa vytvorí supplementary predikát.

Tým sa top-down informácia o väzbách preloží do bottom-up programu.

2.18.3 Výhoda zovšeobecnenej magickej transformácie

Výhodou je, že bottom-up výpočet sa správa podobne ako top-down výpočet, ale zachováva relačný charakter vyhodnocovania.

Zovšeobecnená magická transformácia je teda vhodná najmä pre:

- rekurzívne dotazy,
- programy s viacerými možnými binding patterns,
- programy s funkčnými symbolmi,
- prípady, kde pôvodný bottom-up výpočet neefektívne počíta priveľa faktov,
- prípady, kde pôvodný bottom-up výpočet nemusí terminovať, ale dotazovo obmedzený magický program terminovať môže.

2.19 Magická transformácia pri funkčných symboloch

Pri programoch s funkčnými symbolmi môže bottom-up výpočet pôvodného programu generovať nekonečne veľa termov.

Napríklad pri syntaktickej analýze výrazov môže pôvodný program generovať ľubovoľne veľké výrazy, aj keď dotaz obsahuje konkrétny výraz.

Magická transformácia vytvorí filtre, ktoré do výpočtu vpustia iba podvýrazy zadaného výrazu. Tým sa výpočet môže stať konečným.

Intuícia:

$$m_expr(E)$$

hovorí, že výraz E je relevantný pre aktuálny dotaz.

Podobne:

$$m_term(T)$$

hovorí, že term T je relevantný ako podčasť skúmaného výrazu.

Magic predikáty teda obmedzujú výpočet na relevantné podštruktúry dotazu.

2.20 Krátke porovnanie

Bez magickej transformácie	S magicou transformáciou
počíta celú IDB reláciu	počíta iba relevantnú časť
ignoruje väzby z dotazu	používa binding patterns
bottom-up môže byť neefektívny	bottom-up simuluje top-down filtre
pri funkčných symboloch nemusí terminovať	dotazovo obmedzený výpočet môže terminovať
jednoduchší program	väčší, ale optimalizovaný program

2.21 Krátke zhrnutie na skúšku

Rekurzívne dotazy sa v Datalogu vyhodnocujú pomocou pevného bodu. Naivná evaluácia opakovane aplikuje všetky pravidlá na všetky známe fakty, čo vedie k veľkému množstvu zbytočnej práce. Seminaivná evaluácia optimalizuje výpočet tak, že v každej iterácii používa iba nové fakty z predchádzajúcej iterácie, označované ako Δ relácie.

Datalog s funkčnými symbolmi môže generovať nekonečne veľa termov, napríklad $f(a), f(f(a)), \dots$, a preto nemusí terminovať. Základným mechanizmom pri výpočte je unifikácia termov, teda hľadanie substitúcie, ktorá zjednotí dva termy. Najdôležitejšia je najvšeobecnejšia unifikácia, MGU. Korektná unifikácia má používať occurs-check, aby nepripustila napríklad $X = f(X)$.

Prolog používa top-down výpočet cez SLD rezolúciu. Začína dotazom, vyberá podciele zľava doprava, hľadá pravidlá, ktorých hlavy sa unifikujú s cieľom, a nahrádza cieľ telom pravidla. Pri zlyhaní používa backtracking.

Rule-Goal Tree reprezentuje top-down výpočet ako strom cieľov a pravidiel. Pri rekurzii však môže byť nekonečný. Rule-Goal Graph je konečná grafová reprezentácia, ktorá používa binding patterns, teda informáciu o tom, ktoré argumenty sú viazané a ktoré voľné. Väzby sa prenášajú cez magic predicates a supplementary relations, čím sa výpočet obmedzuje iba na relevantné fakty.

Magická transformácia je optimalizácia rekurzívnych Datalogových dotazov. Pre daný dotaz transformuje pôvodný program na magický program, ktorý dáva rovnaký výsledok, ale bottom-up výpočet počíta iba relevantné fakty podobne ako top-down výpočet. Základom sú magic predikáty a supplementary predikáty. Magic predikáty nesú viazané hodnoty z dotazu a slúžia ako filtre. Supplementary predikáty prenášajú väzby zľava doprava cez telo pravidla.

Zovšeobecnená magická transformácia používa Rule-Goal Graph a binding patterns. Pre každý potrebný ozdobený IDB predikát vytvorí magic predikát a pre uzly pravidiel supplementary predikáty. Umožňuje optimalizovať aj zložitejšie rekurzívne programy, programy s viacerými vzormi väzieb a programy s funkčnými symbolmi.

3 Optimalizácia nerekurzívnych dotazov

3.1 Cieľ optimalizácie nerekurzívnych dotazov

Nerekurzívny dotaz je dotaz, ktorého vyhodnotenie nevyžaduje výpočet pevného bodu. Typicky ide o SPJ dotazy, teda dotazy zložené zo selekcie, projekcie a joinov:

$$\pi_A(\sigma_F(R_1 \times \dots \times R_n)).$$

Po transformáciách možno takýto dotaz chápať ako:

$$\pi_A(R_1 \bowtie \dots \bowtie R_n \bowtie F_1 \bowtie \dots \bowtie F_k).$$

Cieľom optimalizácie je nájst ekvivalentný výraz alebo plán výpočtu, ktorý má nižšie náklady. Optimalizácia sa snaží hlavne:

- vykonať selekcie čo najskôr,
- posúvať projekcie čo najnižšie,
- vyhýbať sa kartézskym súčinom,
- zvoliť vhodné poradie joinov,
- využiť indexy,
- zmenšiť medzivýsledky,
- odstrániť redundantné časti dotazu.

3.2 Strom relačného výrazu

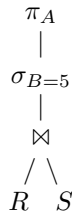
Strom relačného výrazu je reprezentácia výrazu relačnej algebry. Listy stromu sú vstupné relácie a vnútorné uzly sú operácie relačnej algebry, napríklad:

$$\sigma, \pi, \bowtie, \times, \cup.$$

Príklad:

$$\pi_A(\sigma_{B=5}(R \bowtie S)).$$

Jeho strom má koreň π_A , pod ním selekciu $\sigma_{B=5}$ a pod ňou join relácií R a S .



Optimalizácia stromu využíva zákony relačnej algebry. Napríklad:

$$\sigma_{F \wedge G}(R) \equiv \sigma_F(\sigma_G(R)),$$

$$\pi_A(\pi_B(R)) \equiv \pi_A(R), \quad A \subseteq B.$$

Základné pravidlá sú:

- selekcie rozložiť na jednoduché podmienky,
- selekcie posunúť čo najbližšie k listom,
- projekcie posunúť čo najbližšie k listom,
- kartézske súčiny so selekciou nahradiť joinmi,
- spojiť kaskády selekcií a projekcií.

3.3 Hypergraf dotazu

Pri optimalizácii joinov nás zaujíma, ktoré premenné alebo atribúty sú spoločné medzi reláciami. Na to je vhodná reprezentácia pomocou hypergrafu.

Hypergraf je dvojica:

$$H = (X, E),$$

kde X je množina vrcholov a E je množina hyperhrán, pričom každá hyperhrana je neprázdna podmnožina X .

V databázach:

- vrcholy zodpovedajú premenným alebo atribútom,
- hyperhrany zodpovedajú predikátom alebo reláciám,
- jedna hyperhrana obsahuje všetky premenné, ktoré sa vyskytujú v jednom predikáte.

Pre konjunktívny dotaz:

$$Q(x) \leftarrow R(x, y), S(y, z), T(z)$$

dostaneme hypergraf:

$$X = \{x, y, z\},$$

$$E = \{\{x, y\}, \{y, z\}, \{z\}\}.$$

Relácia $R(x, y)$ tvorí hyperhranu $\{x, y\}$, relácia $S(y, z)$ tvorí hyperhranu $\{y, z\}$ a relácia $T(z)$ tvorí hyperhranu $\{z\}$.

3.4 Transformácia SQL dotazu na štandardný hypergraf

Pri SQL dotaze sa najprv vytvorí hypergraf, v ktorom:

- každá tabuľka alebo predikát tvorí hranu,
- každá premenná alebo atribút tvorí uzol,
- rovnosti z **WHERE** alebo **JOIN** klauzuly sa odstránia zjednotením premenných,
- rovnosti s konštantou sa riešia selekciou,
- ostatné selekčné podmienky sa reprezentujú selekčnými hranami.

V štandardnom hypergrafe sa nevyskytujú samostatné hrany rovnosti. Ak máme podmienku:

$$x = y,$$

vyberieme jedného reprezentanta a všetky výskyty x a y nahradíme týmto reprezentantom.

Ak máme:

$$x = a,$$

kde a je konštanta, nahradíme premennú konštantou a aplikujeme selekciu na príslušnú reláciu.

3.5 Dekompozícia hypergrafu

Dekompozícia hypergrafu sa používa na rozklad dotazu na menšie časti. Ak je hypergraf acyklický, join možno často vyhodnotiť efektívnejšie.

3.5.1 GYO redukcia

GYO redukcia, podľa Graham, Yu a Ozsoyoglu, je algoritmus postupného zjednodušovania hypergrafu.

Opakovane aplikujeme tieto kroky:

1. Ak existujú hyperhrany E_i a E_j také, že

$$E_i \subseteq E_j,$$

odstránime E_i .

2. Ak existuje vrchol x , ktorý sa vyskytuje iba v jednej hyperhrane, odstránime tento vrchol.

Ak hypergraf po GYO redukcii úplne zmizne, nazýva sa **GYO redukovateľný**. V databázovej teórii takéto hypergrafy zodpovedajú acyklickým dotazom.

3.6 Uši hypergrafu

Databázová interpretácia GYO redukcie sa často opisuje cez tzv. uši.

Hyperhrana E je **ucho**, ak:

- je izolovaná, alebo
- existuje iná hyperhrana F , nazývaná svedok uchovitosti, taká, že každý vrchol z $E - F$ sa vyskytuje iba v hrane E .

Redukcia potom spočíva v postupnom odstraňovaní uší. Takáto eliminácia dáva poradie, v ktorom možno dotaz rozložiť a vyhodnocovať.

3.7 Wong-Youssefiho algoritmus

Wong-Youssefiho algoritmus je optimalizačný algoritmus pre jednoduché SPJ dotazy. Vstupom je výraz tvaru:

$$\pi_D \sigma_F(R_1 \times \cdots \times R_n).$$

Najprv sa zostrojí štandardný hypergraf dotazu. Potom sa rekurzívne aplikuje procedúra EVAL, ktorá dekomponuje hypergraf.

Algoritmus používa tri hlavné prípady:

3.7.1 1. Nesúvislé komponenty

Ak sa hypergraf G rozpadne na nesúvislé komponenty:

$$H_1, \dots, H_k,$$

potom:

$$res(G) := res(H_1) \times \dots \times res(H_k).$$

Každá komponenta sa môže vyhodnotiť samostatne.

3.7.2 2. Odstránenie selekčnej hrany

Ak odstránime selekčnú hranu U a dostaneme hypergraf H , potom:

$$res(G) := \sigma_U(res(H)).$$

To znamená, že najprv vyhodnotíme zvyšok dotazu a potom aplikujeme selekciu.

3.7.3 3. Odstránenie relačnej hrany

Ak odstránime relačnú hranu R a hypergraf sa rozpadne na komponenty:

$$H_1, \dots, H_k,$$

potom algoritmus najprv zredukuje ostatné hrany pomocou semijoinov:

$$S := S \ltimes R$$

pre každú relačnú hranu S , ktorá sa pretína s R .

Potom:

$$res(G) := R \bowtie res(H_1) \bowtie \dots \bowtie res(H_k).$$

Toto znižuje veľkosť medzivýsledkov.

3.8 Semijoin

Semijoin relácie R s reláciou S označujeme:

$$R \ltimes S.$$

Výsledkom sú tie n -tice z R , ktoré sa dajú spojiť s aspoň jednou n -ticou zo S .

Formálne:

$$R \ltimes S = \pi_{\text{attr}(R)}(R \bowtie S).$$

Semijoin teda zachováva iba atribúty relácie R , ale používa S ako filter.

3.8.1 Príklad

Nech:

$$R(A, B), \quad S(B, C).$$

Potom:

$$R \ltimes S$$

obsahuje tie dvojice (A, B) z R , pre ktoré existuje aspoň jedna hodnota C taká, že $(B, C) \in S$.

Semijoin sa používa na redukciu vstupných relácií pred výpočtom veľkého joinu.

3.9 Reduktor a úplný reduktor

Reduktor je program pozostávajúci z postupnosti príkazov tvaru:

$$R := R \ltimes S.$$

Jeho cieľom je odstrániť z relácií n -tice, ktoré sa aj tak nemôžu objaviť vo výsledku finálneho joinu.

Úplný reduktor je taký reduktor, ktorý zabezpečí, že do výpočtu joinu nevstúpi žiadna zbytočná n -tica.

To znamená, že po aplikácii úplného reduktora každá zostávajúca n -tica v každej relácii patrí do aspoň jednej výslednej n -tice celého joinu.

3.10 Yannakakisov algoritmus

Yannakakisov algoritmus efektívne vyhodnocuje acyklické konjunktívne dotazy. Používa join tree a semijoinové redukcie.

Postup:

1. Zostrojíme join tree pre acyklický hypergraf dotazu.
2. Vykonáme bottom-up semijoinovú redukciu od listov ku koreňu.
3. Vykonáme top-down semijoinovú redukciu od koreňa k listom.
4. Po týchto redukciách sú relácie zredukované úplným reduktorom.
5. Nakoniec vypočítame join zredukovaných relácií.

Výhoda je, že pre acyklické dotazy algoritmus nespôsobuje veľké zbytočné medzivýsledky. Jeho zložitosť je polynomiálna a často sa uvádza ako lineárna vzhľadom na veľkosť vstupu a výstupu.

3.11 Optimalizácia poradia joinov

Join je komutatívny a asociatívny:

$$R \bowtie S \equiv S \bowtie R,$$

$$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T).$$

Preto existuje veľa možných plánov výpočtu toho istého dotazu.

Príklad:

$$R \bowtie S \bowtie T$$

možno počítať ako:

$$(R \bowtie S) \bowtie T$$

alebo:

$$R \bowtie (S \bowtie T).$$

Tieto plány môžu mať veľmi odlišné náklady, pretože medzivýsledky môžu mať rôznu veľkosť.

3.11.1 Zásady výberu poradia joinov

- Najskôr spájať malé relácie.
- Najskôr aplikovať selektívne podmienky.
- Vyhýbať sa kartézskym súčinom.
- Preferovať joiny, ktoré výrazne znižujú výsledok.
- Využívať indexy.
- Pri acyklických dotazoch využiť join tree.

3.12 Odhad veľkosti výstupu dotazu

Optimalizátor potrebuje odhadnúť veľkosť výstupu ešte pred reálnym výpočtom.

Preto databázový systém udržiava štatistiky v katalógu.

Pre reláciu R sa používajú napríklad:

$$B(R) = \text{počet blokov relácie } R,$$

$$T(R) = \text{počet n-tíc relácie } R,$$

$$V(R, A) = \text{počet rôznych hodnôt atribútu } A,$$

$$MIN(R, A), \quad MAX(R, A).$$

Redukčný faktor dotazu Q možno chápať ako pomer:

$$rf(Q) = \frac{B(Q)}{B(R_1) \cdots B(R_n)}.$$

Pri odhade sa často predpokladá nezávislosť podmienok a rovnomerné rozdelenie hodnôt.

3.12.1 Odhady selekcií

Pre selekciu:

$$R.A = c$$

sa používa:

$$rf(R.A = c) = \frac{1}{V(R, A)}.$$

Odhad počtu n-tíc:

$$T(\sigma_{R.A=c}(R)) \approx T(R) \cdot \frac{1}{V(R, A)}.$$

Pre nerovnosť:

$$R.A > c$$

sa používa:

$$rf(R.A > c) = \frac{MAX(R, A) - c}{MAX(R, A) - MIN(R, A)}.$$

3.12.2 Odhad joinu

Pre joinovú podmienku:

$$R.A = S.B$$

sa často používa:

$$rf(R.A = S.B) = \frac{1}{\max(V(R, A), V(S, B))}.$$

Odhad veľkosti joinu:

$$T(R \bowtie S) \approx T(R) \cdot T(S) \cdot \frac{1}{\max(V(R, A), V(S, B))}.$$

3.13 Použitie indexov

Index umožňuje rýchlo nájsť n-ticu podľa hodnoty atribútu.

Rozlišujeme:

- **clustered index**,
- **non-clustered index**.

3.13.1 Clustered index

Clustered index určuje fyzické usporiadanie n-tíc v relácii. Preto môže byť na jednej tabuľke najviac jeden clustered index.

Je vhodný pre:

- časté rozsahové dotazy,
- časté triedenie podľa daného atribútu,
- joiny, pri ktorých sa oplatí mať dáta fyzicky usporiadané.

3.13.2 Non-clustered index

Non-clustered index je samostatná dátová štruktúra, napríklad hash tabuľka alebo B⁺-strom. Na jednej tabuľke ich môže byť viac.

Je vhodný pre:

- selektívne rovnostné podmienky,
- atribúty s veľkým počtom rôznych hodnôt,
- cudzie kľúče často používané v joinoch.

Nie je vhodné vytvárať index pre atribút s malým počtom hodnôt, ak dotaz vracia veľkú časť tabuľky. Napríklad index na známku, ktorá má iba hodnoty $A, \dots, Fx$, nemusí pomôcť, pretože dotaz môže vrátiť príliš veľa n-tíc.

Všeobecné pravidlo:

Ak dotaz číta viac ako približne 20% tabuľky, index nemusí byť výhodný.

3.14 Dekompozícia databázy

Optimalizácia sa netýka iba jedného dotazu, ale aj návrhu schémy.

3.14.1 Horizontálna dekompozícia

Horizontálna dekompozícia delí tabuľku podľa riadkov.

Napríklad:

$$transcripts(StudentId, CourseId, Semester, Grade)$$

môžeme rozdeliť na:

$$transcriptsA, transcriptsB, \dots, transcriptsF.$$

Ak sa často pýtame na konkrétnu známku, dotaz potom číta iba jednu časť.

3.14.2 Vertikálna dekompozícia

Vertikálna dekompozícia delí tabuľku podľa stĺpcov.

Napríklad:

$$students(StudentId, SName, SAddr, SStatus, Photo, Phone)$$

môžeme rozdeliť na:

$$students1(StudentId, SName),$$
$$students2(StudentId, SStatus, Photo, Phone).$$

Toto je výhodné, ak sa väčšina dotazov pýta iba na malý počet atribútov.

3.14.3 Upozornenie

Dekompozícia a denormalizácia môžu zrýchliť niektoré dotazy, ale zhoršiť aktualizácie a zvýšiť zložitosť schémy. Preto sa nemajú robiť predčasne.

3.15 Pohltenie a ekvivalencia konjunktívnych dotazov

Konjunktívny dotaz má tvar:

$$Q(\bar{x}) \leftarrow A_1, \dots, A_n,$$

kde A_i sú atómy.

Dotaz Q_1 je **pohltený** dotazom Q_2 , označujeme:

$$Q_1 \subseteq Q_2,$$

ak pre každú databázu D platí:

$$Q_1(D) \subseteq Q_2(D).$$

Dotazy sú **ekvivalentné**, ak platí obojstranné pohltenie:

$$Q_1 \equiv Q_2 \iff Q_1 \subseteq Q_2 \wedge Q_2 \subseteq Q_1.$$

3.15.1 Intuícia

Ak $Q_1 \subseteq Q_2$, potom Q_1 je silnejší alebo špecifickejší dotaz. Všetko, čo vráti Q_1 , vráti aj Q_2 .

Príklad:

$$Q_1(x) \leftarrow R(x, y), S(y)$$

je pohltený dotazom:

$$Q_2(x) \leftarrow R(x, y),$$

pretože Q_1 má dodatočnú podmienku $S(y)$.

3.16 Testovanie pohltenia konjunktívnych dotazov

Pre konjunktívne dotazy bez porovnávacích predikátov platí homomorfizmové kritérium.

Nech:

$$Q_1(\bar{x}) \leftarrow body_1$$

a

$$Q_2(\bar{x}) \leftarrow body_2.$$

Potom:

$$Q_1 \subseteq Q_2$$

práve vtedy, keď existuje homomorfizmus z tela Q_2 do tela Q_1 , ktorý zachováva výstupné premenné.

Pozor na smer: na dôkaz

$$Q_1 \subseteq Q_2$$

hľadáme zobrazenie:

$$\text{body}(Q_2) \rightarrow \text{body}(Q_1).$$

3.16.1 Príklad

$$Q_1(x) \leftarrow R(x, y), S(y)$$

$$Q_2(x) \leftarrow R(x, z).$$

Chceme ukázať:

$$Q_1 \subseteq Q_2.$$

Existuje homomorfizmus z tela Q_2 do tela Q_1 :

$$x \mapsto x, \quad z \mapsto y.$$

Atóm:

$$R(x, z)$$

sa zobrazí na:

$$R(x, y),$$

ktorý je v tele Q_1 . Preto:

$$Q_1 \subseteq Q_2.$$

3.17 Petrifikované dotazy

Petrifikácia dotazu znamená, že premenné v dotaze nahradíme novými konštantami. Tieto konštanty reprezentujú konkrétne symbolické hodnoty.

Napríklad dotaz:

$$Q(x) \leftarrow R(x, y), S(y)$$

sa petrifikuje na databázu:

$$D_Q = \{R(a_x, a_y), S(a_y)\}.$$

Výstupná premenná x zodpovedá konštante a_x .

Petrifikovaný dotaz sa používa pri testovaní pohltenia cez kanonickú databázu.

3.18 Kanonická databáza

Kanonická databáza dotazu Q je databáza, ktorá vznikne z tela dotazu tak, že každú premennú nahradíme jedinečnou konštantou.

Pre:

$$Q(x) \leftarrow R(x, y), S(y)$$

je kanonická databáza:

$$D_Q = \{R(c_x, c_y), S(c_y)\}.$$

Potom platí:

$$Q_1 \subseteq Q_2$$

práve vtedy, keď petrifikovaný výsledok dotazu Q_1 patrí do výsledku Q_2 nad kanonickou databázou D_{Q_1} .

Intuícia: kanonická databáza je najmenšia databáza, na ktorej je dotaz Q_1 pravdivý.

3.19 Tablá

Tableau je tabuľková reprezentácia konjunktívneho dotazu. Riadky tablá zodpovedajú atómom alebo reláciám a stĺpce atribútom.

Symbody v tabli môžu byť:

- distingované premenné, ktoré sa objavujú vo výstupe,
- nedistingované premenné, ktoré sú iba existenčne kvantifikované,
- konštanty.

Konjunktívny dotaz:

$$Q(x) \leftarrow R(x, y), S(y, z)$$

možno reprezentovať tablom, kde jeden riadok reprezentuje atóm $R(x, y)$ a druhý riadok atóm $S(y, z)$.

Tableau umožňuje:

- testovať ekvivalenciu dotazov,
- minimalizovať dotazy,
- aplikovať závislosti,
- skúmať slabú ekvivalenciu.

3.20 Vynútenie závislostí

Pri optimalizácii dotazov často poznáme integritné obmedzenia, napríklad:

- funkčné závislosti,
- viac-hodnotové závislosti,
- inklúzne závislosti,
- kľúče.

Vynútenie závislostí znamená aplikovať tieto závislosti na tableau alebo dotaz, aby sme zistili ďalšie rovnosti alebo nové riadky, ktoré musia platiť.

Tento proces sa často nazýva **chase**.

3.20.1 Funkčná závislosť

Ak platí funkčná závislosť:

$$A \rightarrow B,$$

a v tabli existujú dva riadky s rovnakou hodnotou v stĺpci A , musia mať rovnakú hodnotu aj v stĺpci B .

Teda príslušné symboly v stĺpci B sa zjednotia.

3.20.2 Inklúzna závislosť

Ak platí inklúzna závislosť:

$$R[A] \subseteq S[B],$$

potom ku každému relevantnému riadku v R musí existovať zodpovedajúci riadok v S .

Pri chase sa takýto riadok doplní, ak ešte neexistuje.

3.21 Slabá ekvivalencia dotazov

Dva dotazy sú **silne ekvivalentné**, ak dávajú rovnaký výsledok nad všetkými databázami.

Pri existencii závislostí však často požadujeme ekvivalenciu iba nad databázami, ktoré spĺňajú dané závislosti.

Dotazy Q_1 a Q_2 sú **slabo ekvivalentné vzhľadom na množinu závislostí** Σ , ak pre každú databázu D , ktorá spĺňa Σ , platí:

$$Q_1(D) = Q_2(D).$$

Teda:

$$Q_1 \equiv_{\Sigma} Q_2.$$

Slabá ekvivalencia je dôležitá preto, lebo niektoré časti dotazu môžu byť redundantné iba vďaka znalosti integritných obmedzení.

3.22 Minimalizácia tabiel

Minimalizácia tablá znamená odstránenie redundantných riadkov alebo symbolov tak, aby výsledný dotaz ostal ekvivalentný pôvodnému.

Postup:

1. Reprezentujeme dotaz ako tableau.
2. Ak existujú závislosti, aplikujeme chase.
3. Hľadáme riadky, ktoré sú redundantné.
4. Odstránime ich, ak sa zachová ekvivalencia dotazu.

Pri konjunktívnych dotazoch bez závislostí minimalizácia zodpovedá odstráneniu atómu, ktorý je pokrytý homomorfizmom z dotazu do jeho poddotazu.

3.22.1 Príklad

Dotaz:

$$Q(x) \leftarrow R(x, y), R(x, z)$$

je ekvivalentný dotazu:

$$Q'(x) \leftarrow R(x, y).$$

Druhý atóm $R(x, z)$ je redundantný, pretože existencia jedného riadku $R(x, y)$ stačí a premenná z sa nikde inde nepoužíva.

3.23 Prepojenie jednotlivých metód

Optimalizácia nerekurzívnych dotazov má viac úrovní:

- **Syntaktická úroveň:** úprava stromu relačného výrazu pomocou zákonov relačnej algebry.
- **Štruktúrálna úroveň:** reprezentácia dotazu hypergrafom, dekompozícia, GYO redukcia, Wong-Youssefiho algoritmus.
- **Semijoinová úroveň:** reduktory a Yannakakisov algoritmus pre acyklické dotazy.
- **Nákladová úroveň:** odhad veľkostí výstupov, výber poradia joinov a indexov.
- **Logická úroveň:** pohltenie, ekvivalencia, kanonické databázy, tablá, chase a minimalizácia.
- **Fyzická úroveň:** indexy, denormalizácia, horizontálna a vertikálna dekompozícia.

3.24 Krátke zhrnutie na skúšku

Optimalizácia nerekurzívnych dotazov sa snaží nájsť ekvivalentný, ale lacnejší plán výpočtu. Na úrovni relačnej algebry používame strom výrazu a pravidlá ako posúvanie selekcií a projekcií nadol, nahrádzanie kartézskych súčinov joinmi a elimináciu zbytočných operácií.

Na štruktúrálnej úrovni reprezentujeme dotaz hypergrafom, kde uzly sú premenné a hrany sú relácie alebo predikáty. Pomocou GYO redukcie vieme rozpoznať acyklické dotazy. Wong-Youssefiho algoritmus rozkladá SPJ dotaz podľa hypergrafu a používa semijoiny na zmenšovanie relácií.

Semijoin $R \bowtie S$ ponechá v R iba tie n -tice, ktoré sa spájajú s S . Úplný reduktor odstráni všetky n -tice, ktoré sa nemôžu objaviť vo výsledku. Yannakakisov algoritmus pre acyklické dotazy používa bottom-up a top-down semijoinové redukcie a potom vypočíta finálny join.

Pri nákladovej optimalizácii optimalizátor odhaduje veľkosť výsledkov pomocou štatistík ako $T(R)$, $B(R)$ a $V(R, A)$. Poradie joinov sa volí tak, aby

boli medzivýsledky čo najmenšie. Fyzická optimalizácia používa indexy, pričom clustered index určuje fyzické poradie n-tíc a non-clustered index je samostatná pomocná štruktúra.

Logická optimalizácia používa pohltenie a ekvivalenciu konjunktívnych dotazov. Pohltenie $Q_1 \subseteq Q_2$ znamená, že každý výsledok Q_1 je aj výsledkom Q_2 . Testuje sa homomorfizmom alebo cez kanonickú databázu. Tablá sú tabuľkové reprezentácie dotazov. Pomocou vynútenia závislostí, teda chase, vieme zohľadniť funkčné a inklúzne závislosti. Slabá ekvivalencia znamená ekvivalenciu iba nad databázami spĺňajúcimi dané závislosti. Minimalizácia tabiel odstraňuje redundantné riadky a tým zjednodušuje dotaz.

4 Vyššie normálne formy

4.1 Motivácia normalizácie

Cieľom normalizácie je navrhnúť relačnú databázu tak, aby obsahovala čo najmenej redundancie a aby sa minimalizovali aktualizácie, vkladacie a mazacie anomálie.

Ľubovoľnú databázu by sme teoreticky vedeli uložiť do jednej veľkej tabuľky, ale to vedie k problémom:

- redundancia údajov,
- riziko nekonzistencie,
- anomálie pri vkladaní, mazaní a aktualizácii,
- potreba veľkého množstva hodnôt NULL,
- plytvanie pamäťou.

Normalizácia je formálna metodika, ktorá používa závislosti medzi atribútmi na hľadanie vhodnej dekompozície relácií.

4.2 Funkčné závislosti

Nech R je relačná schéma a $X, Y \subseteq R$ sú množiny atribútov. Hovoríme, že v relácii platí **funkčná závislosť**

$$X \rightarrow Y,$$

ak pre každé prípustné naplnenie relácie platí: ak sa ľubovoľné dva riadky zhodujú na atribútoch X , potom sa musia zhodovať aj na atribútoch Y .

Formálne:

$$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y].$$

Intuitívne:

$$X \rightarrow Y$$

znamená, že hodnota atribútov X jednoznačne určuje hodnotu atribútov Y .

4.2.1 Príklad

Majme reláciu:

$$Beer(Bar, Address, Beer, Manufacturer, Price).$$

Môžu platiť závislosti:

$$Bar \rightarrow Address,$$

$$Bar, Beer \rightarrow Price.$$

To znamená, že bar má jednu adresu a dvojica bar–pivo určuje cenu.

Nemusí však platiť:

$$Bar \rightarrow Beer,$$

pretože v jednom bare môžu predávať viac pív.

4.3 Armstrongove axiómy

Funkčné závislosti možno odvodzať pomocou Armstrongových axióm:

1. **Reflexívnosť**

$$Y \subseteq X \Rightarrow X \rightarrow Y.$$

2. **Rozšírenie**

$$X \rightarrow Y \Rightarrow XZ \rightarrow YZ.$$

3. **Tranzitívnosť**

$$X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z.$$

Z nich možno odvodiť ďalšie pravidlá:

$$X \rightarrow Y \wedge X \rightarrow Z \Rightarrow X \rightarrow YZ,$$

$$X \rightarrow Y \wedge WY \rightarrow Z \Rightarrow WX \rightarrow WZ,$$

$$X \rightarrow Y \wedge Z \subseteq Y \Rightarrow X \rightarrow Z.$$

Armstrongove axiómy sú korektné a úplné pre odvodzovanie funkčných závislostí.

4.4 Uzáver množiny atribútov

Nech F je množina funkčných závislostí a X množina atribútov. Uzáver X^+ vzhľadom na F je množina všetkých atribútov, ktoré sú funkčne určené množinou X .

$$X^+ = \{A \mid F \models X \rightarrow A\}.$$

Algoritmus výpočtu:

1. Inicializuj:

$$X^+ := X.$$

2. Opakovane pre každú závislosť $U \rightarrow V \in F$: ak

$$U \subseteq X^+,$$

potom:

$$X^+ := X^+ \cup V.$$

3. Skonči, keď sa už nič nepridáva.

Uzáver sa používa na testovanie, či je X nadkľúčom.

4.5 Kľúče a nadkľúče

Množina atribútov K je **nadkľúč** relácie R , ak:

$$K \rightarrow R.$$

Teda:

$$K^+ = R.$$

Kľúč je minimálny nadkľúč vzhľadom na množinovú inklúziu. To znamená, že K je kľúč, ak:

$$K^+ = R$$

a pre každé $A \in K$ platí:

$$(K - \{A\})^+ \neq R.$$

Atribút, ktorý patrí do niektorého kľúča, sa nazýva **kľúčový atribút**. Ostatné atribúty sú **neklúčové**.

4.6 Tretia normálna forma

Relačná schéma R je v **tretej normálnej forme**, skratene 3NF, vzhľadom na množinu funkčných závislostí F , ak pre každú netriviálnu (takú, že ľavá strana sa nedá skrátiť) funkčnú závislosť

$$X \rightarrow Y$$

z F^+ platí aspoň jedna z možností:

1. X je nadkľúč,
2. Y je kľúčový atribút,
3. závislosť je triviálna, teda $Y \in X$.

3NF teda povoľuje niektoré závislosti, ktoré by BCNF zakázala, ak je pravá strana kľúčovým atribútom.

4.6.1 Výhoda 3NF

Pre 3NF existuje algoritmus dekompozície, ktorý garantuje:

- bezstratový join,
- zachovanie funkčných závislostí.

Preto je 3NF často prakticky výhodná.

4.7 Boyce-Coddova normálna forma

Relačná schéma R je v **Boyce-Coddovej normálnej forme**, skrátené BCNF, ak pre každú netriviálnu funkčnú závislosť

$$X \rightarrow Y$$

z F^+ platí:

X je nadkľúč.

BCNF je silnejšia ako 3NF:

$$BCNF \Rightarrow 3NF.$$

Ak je schéma v BCNF, neobsahuje redundanciu vzhľadom na funkčné závislosti. Nevýhodou je, že dekompozícia do BCNF nemusí zachovať všetky funkčné závislosti. Materiály priamo uvádzajú, že bezstratový rozklad do nejakej BCNF sa dá nájsť v polynomiálnom čase, ale negarantuje zachovanie všetkých funkčných závislostí.

4.8 Dekompozícia do BCNF

Ak relácia R nie je v BCNF, existuje netriviálna funkčná závislosť:

$$X \rightarrow Y,$$

kde X nie je nadkľúč.

Potom reláciu rozložíme na:

$$R_1 = X \cup Y,$$

$$R_2 = R - Y.$$

Tento rozklad je bezstratový.

Algoritmus:

1. Nájsť funkčnú závislosť, ktorá porušuje BCNF.
2. Minimalizuj jej ľavú stranu
3. nech výsledok je $X \rightarrow Y$

4. Rozlož reláciu na XY a $R - Y$.
5. Pokračuj, kým všetky relácie nie sú v BCNF.

V praktickom návrhu je prirodzené začať dobrou 3NF dekompozíciou so zachovaním závislostí a potom sa snažiť jednotlivé relácie posunúť do BCNF, ak sa tým nezničia dôležité závislosti.

4.9 Bezstratová dekompozícia

Dekompozícia relácie R na R_1 a R_2 je **bezstratová**, ak pre každú platnú inštanciu r platí:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r).$$

Pre rozklad do dvoch relácií platí jednoduchý test:

Dekompozícia $R \rightarrow R_1, R_2$ je bezstratová, ak:

$$(R_1 \cap R_2) \rightarrow R_1$$

alebo:

$$(R_1 \cap R_2) \rightarrow R_2.$$

Inými slovami, spoločné atribúty musia byť nadkľúčom aspoň v jednej z dvoch relácií.

Pre rozklad do troch a viac relácií tento jednoduchý test nestačí a používa sa **chase**.

4.10 Multizávislosti

Funkčná závislosť hovorí, že jedna hodnota určuje inú hodnotu. **Multizávislosť** hovorí, že jedna množina atribútov nezávisle určuje množinu hodnôt inej skupiny atribútov.

Nech:

$$R = X \cup Y \cup Z.$$

V relácii R platí multizávislosť:

$$X \twoheadrightarrow Y,$$

ak pre každé dve n-tice:

$$r(X, Y_1, Z_1), \quad r(X, Y_2, Z_2)$$

musí v relácii existovať aj:

$$r(X, Y_1, Z_2), \quad r(X, Y_2, Z_1).$$

Teda pri pevnej hodnote X sú hodnoty Y nezávislé od hodnôt Z .

4.10.1 Príklad

Majme reláciu:

$$emp(EMP_name, Proj_name, Boss_name).$$

Ak zamestnanec môže pracovať na viacerých projektoch a zároveň môže mať viacerých nadriadených, pričom projekty a nadriadení sú nezávislé informácie, potom platí:

$$EMP_name \twoheadrightarrow Proj_name,$$

$$EMP_name \twoheadrightarrow Boss_name.$$

Pre zamestnanca **joe** a projekty x, y a šéfov **fred, lisa** musí relácia obsahovať všetky kombinácie:

$$(joe, x, fred), (joe, y, lisa), (joe, x, lisa), (joe, y, fred).$$

Takáto relácia obsahuje redundanciu a má sa rozložiť na:

$$project(EMP_name, Proj_name),$$

$$boss(EMP_name, Boss_name).$$

Tento príklad presne ilustruje materiál k multizávislostiam a 4NF.

4.11 Triviálna multizávislosť

Multizávislosť:

$$X \twoheadrightarrow Y$$

je triviálna, ak:

$$Y \subseteq X$$

alebo:

$$X \cup Y = R.$$

Triviálne multizávislosti nespôsobujú problém pri normalizácii.

4.12 Štvrtá normálna forma

Relačná schéma R je vo **štvrtej normálnej forme**, skrátené 4NF, ak pre každú netriviálnu multizávislosť:

$$X \twoheadrightarrow Y$$

z uzáveru závislostí F^+ platí:

$$X \text{ je nadkľúč.}$$

4NF je silnejšia ako BCNF:

$$4NF \Rightarrow BCNF.$$

Dôvod je, že každá funkčná závislosť:

$$X \rightarrow Y$$

implikuje aj multizávislosť:

$$X \twoheadrightarrow Y.$$

Preto 4NF rieši nielen funkčné závislosti, ale aj nezávislé viacnásobné hodnoty.

4.13 Dekompozícia do 4NF

Ak relácia R porušuje 4NF kvôli netriviálnej multizávislosti:

$$X \twoheadrightarrow Y,$$

kde X nie je nadkľúč, rozložíme R na:

$$R_1 = X \cup Y,$$

$$R_2 = R - Y.$$

Tento rozklad je bezstratový vzhľadom na danú multizávislosť.

Naivný algoritmus 4NF:

1. Začni s množinou relácií $D = \{R\}$.
2. Kým existuje relácia $S \in D$, ktorá nie je v 4NF:
 - (a) nájdi netriviálnu multizávislosť $X \twoheadrightarrow Y$, ktorá porušuje 4NF,
 - (b) nahraď S reláciami:

$$S - Y$$

a

$$X \cup Y.$$

Materiály zdôrazňujú, že tento algoritmus garantuje bezstratový join, ale pri vyšších normálnych formách treba byť opatrný, pretože návrh môže smerovať k príliš jemnej dekompozícii.

4.14 Joinovacie závislosti

Joinovacia závislosť, anglicky *join dependency*, je zovšeobecnenie multizávislosti.

Nech $R, R_1, \dots, R_n$ sú množiny atribútov. Joinovacia závislosť má tvar:

$$JD(R_1, \dots, R_n)$$

a hovorí, že pre každé platné naplnenie relácie r nad atribútmi R platí:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_n}(r).$$

Teda relácia sa dá bezstratovo zrekonštruovať z projekcií na množiny atribútov $R_1, \dots, R_n$.

Joinovacia závislosť je všeobecná podmienka bezstratovej dekompozície. Multizávislosť je špeciálny prípad binárnej joinovacej závislosti.

4.15 Piata normálna forma

Relačná schéma R je v **piatej normálnej forme**, skratene 5NF, alebo **PJNF** – *project-join normal form*, ak pre každú netriviálnu joinováciu závislosť:

$$JD(R_1, \dots, R_n)$$

z F^+ platí, že každý komponent R_i je nadkľúčom relácie R .

Intuitívne: relácia je v 5NF, ak už neexistuje netriviálny bezstratový rozklad na menšie relácie, ktorý by nebol vynútený kľúčmi.

Platí hierarchia:

$$5NF \Rightarrow 4NF \Rightarrow BCNF \Rightarrow 3NF.$$

5NF zohľadňuje nielen funkčné závislosti a multizávislosti, ale aj všeobecné joinovacie závislosti.

4.16 Inklúzne závislosti

Inklúzna závislosť vyjadruje, že hodnoty určitej množiny atribútov jednej relácie sa musia nachádzať ako hodnoty určitej množiny atribútov inej relácie.

Formálne:

$$R[A_1, \dots, A_k] \subseteq S[B_1, \dots, B_k].$$

To znamená, že pre každú n -ticu $t \in R$ existuje n -tica $u \in S$ taká, že:

$$t[A_1, \dots, A_k] = u[B_1, \dots, B_k].$$

4.16.1 Príklad

Majme:

$$\begin{aligned} &students(Student_ID, Name), \\ &grades(Student_ID, Course_ID, Grade). \end{aligned}$$

Ak sa `Student_ID` objaví v relácii `grades`, musí existovať aj v relácii `students`. Teda:

$$grades[Student_ID] \subseteq students[Student_ID].$$

Cudzí kľúč je špeciálny prípad inklúznej závislosti. Materiály uvádzajú presne tento univerzitný príklad a zdôrazňujú, že inklúzne závislosti zovšeobecňujú koncept cudzích kľúčov a slabých entít.

4.17 DKNF

DKNF, teda *domain-key normal form*, je doménovo-kľúčová normálna forma.

Relácia je v DKNF, ak každé obmedzenie nad reláciou je logickým dôsledkom:

- domén atribútov,
- kľúčov.

Formálne:

každý constraint $\models$ domény a kľúče.

Doména je množina povolených hodnôt atribútu. Constraint je ľubovoľné presné pravidlo, pri ktorom vieme rozhodnúť, či je splnené.

DKNF je najsilnejšia normálna forma z uvedených:

$$DKNF \Rightarrow 5NF \Rightarrow 4NF \Rightarrow BCNF \Rightarrow 3NF.$$

Materiály ju označujú ako "ultimate normal form", pretože ak je relácia v DKNF, všetky lokálne obmedzenia sú vysvetlené iba doménami a kľúčmi.

4.17.1 Problém DKNF

DKNF je teoreticky veľmi pekná, ale prakticky problematická:

- neexistuje univerzálny algoritmus, ktorý by zostrojil DKNF dekompozíciu,
- všeobecne nemožno ani rozhodnúť, či DKNF dekompozícia existuje,
- neexistuje univerzálny recept na overenie, či daná dekompozícia je v DKNF.

Preto sa v praxi pracuje najmä s 3NF, BCNF, prípadne 4NF a 5NF pri špecifických typoch redundancie.

4.18 Metodika normalizácie

Praktická metodika návrhu databázy môže vyzeráť takto:

1. Zisti atribúty a závislosti.

Najdôležitejšie je dobre pochopiť doménu. Produktom logického návrhu nie sú len tabuľky, ale hlavne atribúty a závislosti medzi nimi.

2. Urči funkčné závislosti.

Napríklad:

$$StudentID \rightarrow Name, Program,$$

$$CourseID \rightarrow CourseName.$$

3. Spočítaj uzávery atribútov a nájdi kľúče.

Pre kandidáta K spočítaj K^+ . Ak:

$$K^+ = R,$$

potom K je nadkľúč. Ak je minimálny, je to kľúč.

4. Vytvor 3NF dekompozíciu.

3NF je praktický základ, pretože vieme dosiahnuť bezstratový rozklad so zachovaním funkčných závislostí.

5. Skús posun do BCNF.

Over, či relácie neporušujú BCNF. Ak áno, rozkladaj podľa porušujúcich závislostí:

$$X \rightarrow Y.$$

Treba však kontrolovať, či nestratíme dôležité funkčné závislosti.

6. Hľadaj multizávislosti.

Ak existujú nezávislé viacnásobné hodnoty, napríklad zamestnanec má nezávisle projekty a šéfov, treba uvažovať o 4NF.

7. Hľadaj joinovacie závislosti.

Ak relácia obsahuje redundanciu, ktorú neodhalia funkčné závislosti ani multizávislosti, treba uvažovať o 5NF.

8. Skontroluj inklúzne závislosti.

Tie typicky vyjadrujú cudzie kľúče a referenčnú integritu.

9. Zastav sa včas.

Vyššie normálne formy môžu viesť k príliš veľkému počtu malých relácií. To môže zhoršiť čitateľnosť, výkon aj údržbu systému.

Materiály výslovne upozorňujú, že normalizáciu treba používať opatrne: vyššie normálne formy síce odstraňujú ďalšie typy redundancie, ale konštrukcia 4NF a vyšších môže mať exponenciálnu zložitosť a proces môže smerovať až k rozkladu do binárnych relácií, čo zvyčajne nechceme. Prakticky zdravý počiatočný návrh je často BCNF vyrobená z 3NF, ktorá zachováva čo najviac funkčných závislostí.

4.19 Vzťah normálnych foriem

Normálne formy tvoria hierarchiu:

$$DKNF \Rightarrow 5NF \Rightarrow 4NF \Rightarrow BCNF \Rightarrow 3NF.$$

Forma	Rieši závislosti	Podmienka
3NF	funkčné závislosti	determinant je nadkľúč alebo pravá strana je kľúčový atribút
BCNF	funkčné závislosti	každý determinant je nadkľúč
4NF	multizávislosti	pri každej netriviálnej MVD je ľavá strana nadkľúč
5NF	joinovacie závislosti	netriviálne JD sú vynútené kľúčmi
DKNF	všetky lokálne obmedzenia	všetko vyplýva z domén a kľúčov

4.20 Krátke zhrnutie na skúšku

Normalizácia je formálna metóda návrhu relačnej databázy, ktorej cieľom je odstrániť redundanciu a anomálie. Základom sú funkčné závislosti $X \rightarrow Y$, ktoré hovoria, že hodnota X jednoznačne určuje hodnotu Y . Pomocou uzáverov atribútov vieme určovať nadklúče a klúče.

3NF povoľuje závislosť $X \rightarrow A$, ak X je nadklúč alebo A je kľúčový atribút. BCNF je silnejšia: každá netriviálna funkčná závislosť musí mať na ľavej strane nadklúč. BCNF odstraňuje redundanciu vzhľadom na funkčné závislosti, ale nemusí zachovať všetky závislosti.

4NF rieši multizávislosti $X \twoheadrightarrow Y$. Tie vznikajú, keď pri pevnom X existujú nezávislé množiny hodnôt Y a Z . Relácia je v 4NF, ak každá netriviálna multizávislosť má na ľavej strane nadklúč.

5NF rieši všeobecné joinovacie závislosti. Relácia je v 5NF, ak každá netriviálna joinovacia závislosť je vynútená kľúčmi. Inklúzne závislosti vyjadrujú, že hodnoty jedného atribútu alebo skupiny atribútov sa musia nachádzať v inej relácii; cudzí kľúč je ich špeciálny prípad.

DKNF je najsilnejšia forma. Relácia je v DKNF, ak každé obmedzenie vyplýva iba z domén a kľúčov. Prakticky je však ťažko použiteľná, pretože neexistuje univerzálny algoritmus na jej dosiahnutie.

V praxi sa odporúča začať dobrou 3NF dekompozíciou, ktorá zachováva funkčné závislosti, potom sa snažiť prejsť do BCNF, ak tým nestratíme dôležité závislosti, a vyššie formy použiť len vtedy, keď existujú reálne multizávislosti alebo joinovacie závislosti.

5 Distribúované databázy

5.1 Model distribuovanej databázy

Distribuovaná databáza je databázový systém, v ktorom sú dáta, výpočty alebo správa transakcií rozdelené medzi viacerými uzlami, nazývanými **sites**.

Každý uzel môže obsahovať:

- lokálny databázový systém,
- lokálne dáta,
- lokálny log,
- správcu uzla, tzv. **site manager**.

Transakcia môže začať na jednom uzle, ale počas svojho vykonania môže čítať alebo meniť dáta uložené na viacerých uzloch.

5.1.1 Centralizovaná architektúra

V centralizovanej architektúre existuje jeden databázový server, ku ktorému pristupujú všetky transakcie.

Výhody:

- jednoduchosť,
- dlhodobo otestované riešenie,
- jednoduchšia správa.

Nevýhody:

- server je úzke hrdlo,
- nízka škálovateľnosť,
- pri výpadku servera sú dáta nedostupné.

5.1.2 Distribúovaná architektúra

V distribúovanej architektúre existuje viacero uzlov:

$$Site_1, Site_2, \dots, Site_n.$$

Každý uzol má vlastný databázový systém a komunikuje s ostatnými uzlami cez správcu uzla.

Požiadavky na distribúovanú databázu:

- **ACID vlastnosti**,
- **transparentnosť** – klient nemá vidieť, či je systém distribúovaný,
- **odolnosť voči zlyhaniu** – jeden server nesmie donekonečna čakať na obnovu iného servera.

5.2 ACID vlastnosti

Distribúovaná databáza musí zachovať štandardné vlastnosti transakcií:

- **Atomicity** – transakcia sa vykoná celá alebo vôbec.
- **Consistency** – transakcia prevedie konzistentnú databázu opäť do konzistentného stavu.
- **Isolation** – paralelné transakcie sa musia správať tak, ako keby sa vykonali sériovo.
- **Durability** – po potvrdení transakcie jej zmeny prežijú aj výpadky systému.

V distribúovanom prostredí je najťažšie zabezpečiť atomicitu a izoláciu, pretože jedna transakcia môže zasahovať viacero uzlov.

5.3 Atomický commit

Atomický commit rieši problém, ako rozhodnúť o potvrdení alebo zrušení transakcie, ktorá sa vykonáva na viacerých uzloch.

Najdôležitejšia požiadavka je:

buď všetky uzly rozhodnú COMMIT, alebo všetky uzly rozhodnú ABORT.

Nie je prípustné, aby jeden uzol transakciu potvrdil a iný ju zrušil.

5.4 Požiadavky na Atomic Commit Protocol

Atomic Commit Protocol, skrátene ACP, musí spĺňať tieto požiadavky:

1. Všetky uzly, ktoré rozhodnú, musia rozhodnúť rovnako:

COMMIT alebo *ABORT*.

2. Ak už uzol rozhodol, nesmie svoje rozhodnutie zmeniť.
3. Rozhodnutie COMMIT môže byť prijaté iba vtedy, ak všetky uzly hlasovali YES.
4. Ak všetky uzly hlasovali YES a nenastali výpadky, všetky uzly musia nakoniec rozhodnúť COMMIT.
5. Ak dostatočne dlho nenastávajú výpadky, všetky nezlyhané uzly musia nakoniec rozhodnúť.

Predpokladá sa, že uzly môžu zlyhať zastavením, ale nerobia byzantské chyby, teda neposielajú úmyselne nesprávne správy.

5.5 Dvojfázový Atomic Commit Protocol

Dvojfázový atomic commit protokol, skrátene **2ACP** alebo 2PC, má jedného koordinátora a viacero účastníkov.

5.5.1 Fáza 1: hlasovanie

Transakcia požiada koordinátora o commit.

Ak koordinátor sám nemôže commitnúť, okamžite rozhodne:

ABORT.

Inak:

1. koordinátor zapíše do logu:

$\langle prepare\ T \rangle$,

2. pošle všetkým účastníkom správu:

$[VOTE\ T]$,

3. každý účastník zistí, či vie transakciu potvrdiť,

4. ak nie, zapíše:

$\langle ABORT\ T \rangle$

a odpovie:

$[NO\ T]$,

5. ak áno, zapíše:

$\langle YES\ T \rangle$

a odpovie:

$[YES\ T]$.

5.5.2 Fáza 2: rozhodnutie

Koordinátor čaká na odpovede.

Ak dostane aspoň jedno:

$[NO\ T]$,

rozhodne:

$ABORT$.

Zapíše:

$\langle ABORT\ T \rangle$

a pošle všetkým účastníkom:

$[ABORT\ T]$.

Ak dostane od všetkých účastníkov:

$[YES\ T]$,

rozhodne:

$COMMIT$.

Zapíše:

$\langle COMMIT\ T \rangle$

a pošle:

$[COMMIT\ T]$.

Účastníci, ktorí hlasovali **YES**, čakajú na konečné rozhodnutie koordinátora.

5.5.3 Nevýhoda 2ACP

Hlavný problém 2ACP je **blokovanie**. Ak účastník zahlasoval **YES** a následne zlyhá koordinátor, účastník nevie, či má urobiť **COMMIT** alebo **ABORT**.

Musí čakať na obnovu koordinátora alebo na informáciu od iného uzla.

5.6 Obnova pri 2ACP

Pri obnove účastník číta svoj log:

- ak nájde:

$\langle COMMIT\ T \rangle$,

vykoná **redo**;

- ak nájde:

$\langle ABORT\ T \rangle$

alebo:

$\langle NO\ T \rangle$,

vykoná **undo**;

- ak nájde:

$\langle YES\ T \rangle$,

kontaktuje koordinátora a čaká na rozhodnutie.

Koordinátor po obnove tiež číta log. Ak nájde **COMMIT**, opäť rozpošle commit. Inak môže rozposlať abort.

5.7 Trojfázový Atomic Commit Protocol

Trojfázový atomic commit protokol, skrátene **3ACP**, pridáva medzi hlasovanie a konečné rozhodnutie novú fázu:

PRECOMMIT.

Fázy sú:

1. **CanCommit / Vote**
2. **PreCommit**
3. **DoCommit**

5.7.1 Myšlienka 3ACP

Cieľom 3ACP je znížiť blokovanie pri výpadku koordinátora.

Ak koordinátor zlyhá, účastníci si môžu z informácie o stave **PRECOMMIT** odvodiť, aké rozhodnutie je bezpečné:

- ak nikto nedostal **PRECOMMIT**, bezpečné rozhodnutie je **ABORT**,
- ak aspoň jeden nezlyhaný účastník dostal **PRECOMMIT**, nový koordinátor môže pokračovať ku **COMMIT**.

5.7.2 Priebek 3ACP

Ak všetci účastníci hlasujú **YES**, koordinátor neposiela hneď **COMMIT**, ale najprv pošle:

[*PRECOMMIT*].

Účastníci odpovedia:

[*ACK*].

Až keď koordinátor dostane potvrdenia, pošle:

[*COMMIT*].

Ak niektorý účastník hlasuje **NO**, rozhodne sa:

ABORT.

5.7.3 Výhoda a obmedzenie 3ACP

3ACP je menej blokujúci než 2ACP pri výpadku koordinátora.

Ak však môžu zlyhávať aj komunikačné linky, teda môže dôjsť k rozdeleniu siete, ani 3ACP nemôže úplne zabrániť blokovaniu. Pri sieťových partíciách musí protokol používať väčšinové rozhodovanie alebo quorum.

5.8 Výber koordinátora

Ak koordinátor zlyhá, treba zvoliť nového koordinátora. Cieľom je, aby všetky nezlyhané uzly súhlasili na tom, kto je nový koordinátor.

Predpoklady:

- každý uzol má jednoznačný identifikátor,
- každý uzol pozná identifikátory ostatných uzlov,
- cieľom je vybrať nezlyhaný uzol s najväčším identifikátorom.

5.9 Bully algoritmus

Bully algoritmus volí za koordinátora živý uzol s najväčším identifikátorom.

Postup:

1. Uzol, ktorý zistí výpadok koordinátora, pošle správu

ELECTION

všetkým uzlom s vyšším identifikátorom.

2. Ak nikto neodpovie, tento uzol sa stane koordinátorom.

3. Ak niektorý vyšší uzol odpovie:

OK,

pôvodný uzol prestane kandidovať a čaká.

4. Vyšší uzol spustí vlastnú voľbu.

5. Nakoniec najvyšší dostupný uzol oznámi:

COORDINATOR.

Volá sa *bully*, pretože uzol s vyšším identifikátorom vždy preberie voľbu od nižších uzlov.

5.10 Replikácia dát

Pri replikácii je jeden dátový objekt uložený na viacerých uzloch:

$$x_1, x_2, \dots, x_n.$$

Výhody replikácie:

- vyššia dostupnosť,
- rýchlejšie čítanie,
- odolnosť voči výpadkom.

Nevýhody:

- zložitejšie zápisy,
- nutnosť udržať konzistenciu kópií,
- zložitejšie zamykanie a obnova.

Cieľom je zachovať izoláciu transakcií aj vtedy, keď existuje viac kópií rovnakého objektu.

5.11 Distribuované zamykanie

Distribuované zamykanie rozširuje dvojfázové zamykanie do prostredia viacerých uzlov.

5.11.1 Centralizovaný správca zámkov

Jeden uzol spravuje všetky zámky.

Výhody:

- jednoduchá implementácia,
- jednoduché zisťovanie konfliktov.

Nevýhody:

- správca zámkov je úzke hrdlo,
- pri jeho výpadku systém nemôže pokračovať.

5.11.2 Primary-copy schéma

Každý dátový objekt má jednu primárnu kópiu. Uzol s primárnou kópiou spravuje zámky pre daný objekt.

Pravidlá:

- read-lock možno získať z ľubovoľného uzla s replikou,
- write-lock sa musí získať od správcu primárnej kópie,
- správca primárnej kópie musí následne zabezpečiť zápis aj do ostatných replík.

Výhoda je, že čítanie nemá jedno centrálné úzke hrdlo. Nevýhoda je, že výpadok správcu primárnej kópie spôsobuje problém pre zápisy daného objektu.

5.12 Quorum protokol

Quorum protokol je kompromis medzi prístupom read-one-write-all a väčšinovým protokolom.

Každému uzlu priradíme váhu:

$$w_i > 0.$$

Nech:

$$S = \sum_i w_i.$$

Zvolíme:

$$Q_r$$

ako read quorum a:

$$Q_w$$

ako write quorum tak, aby platilo:

$$Q_r + Q_w > S$$

a:

$$2Q_w > S.$$

Pravidlá:

- na čítanie treba získať zámky na uzloch s celkovou váhou aspoň

$$Q_r,$$

- na zápis treba získať zámky na uzloch s celkovou váhou aspoň

$$Q_w.$$

Podmienka:

$$Q_r + Q_w > S$$

zabezpečuje, že každé čítacie quorum sa pretína s každým zápisovým quorum.

Podmienka:

$$2Q_w > S$$

zabezpečuje, že dve zápisové quorum sa vždy pretínajú.

Tým sa zabráni tomu, aby dve transakcie nezávisle zapisovali rôzne kópie bez vzájomného konfliktu.

5.13 Distribuované deadlocky

V distribuovanom systéme môže vzniknúť deadlock, ktorý nie je viditeľný lokálne na jednom uzle.

Každý uzol udržiava lokálny wait-for graph:

$$WFG_i.$$

Vrcholy sú transakcie a hrana:

$$T_i \rightarrow T_j$$

znamená, že transakcia T_i čaká na transakciu T_j .

Globálny deadlock existuje, ak v globálnom wait-for grafe existuje cyklus.

5.13.1 Detekcia distribuovaného deadlocku

Postup:

1. koordinátor si vyžiada lokálne wait-for grafy,
2. spojí ich do aproximácie globálneho wait-for grafu,
3. ak nájde cyklus, vyberie transakciu na abort,
4. abort sa vykoná pomocou atomic commit protokolu.

Problém je, že kvôli oneskoreniam správ koordinátor nemusí poznať presný aktuálny globálny graf. Môže preto detegovať aj cyklus, ktorý už v skutočnosti neexistuje. Následkom môže byť zbytočný abort transakcie.

5.14 Synchronizácia fyzických hodín

V distribuovanom systéme má každý uzol vlastné fyzické hodiny:

$$C_i(t).$$

Tieto hodiny sa môžu líšiť, pretože:

- nie sú spustené úplne naraz,
- majú rôzne odchýlky,
- rôzne rýchlo driftujú.

Cieľom synchronizácie fyzických hodín je zabezpečiť, aby rozdiel medzi hodinami uzlov bol čo najmenší:

$$|C_i(t) - C_j(t)| \leq \epsilon.$$

5.15 Christianov protokol

Christianov protokol synchronizuje klienta s časovým serverom.

Postup:

1. Klient pošle serveru požiadavku v čase:

$$T_0.$$

2. Server odpovie svojím aktuálnym časom:

$$T_s.$$

3. Klient prijme odpoveď v čase:

$$T_1.$$

4. Klient odhadne oneskorenie ako polovicu round-trip time:

$$\frac{T_1 - T_0}{2}.$$

5. Nastaví čas približne na:

$$T_s + \frac{T_1 - T_0}{2}.$$

Christianov protokol predpokladá približne symetrické oneskorenia správ.

5.16 Berkeley protokol

Berkeley protokol nepoužíva externý presný časový server. Jeden uzol je zvolený ako master a synchronizuje ostatné uzly relatívne voči sebe.

Postup:

1. Master sa opýta ostatných uzlov na ich lokálny čas.
2. Zistí rozdiely medzi hodinami.
3. Vypočíta priemerný čas alebo priemernú korekciu.
4. Pošle každému uzlu korekciu, teda o koľko má svoje hodiny posunúť.

Výhoda Berkeleyho protokolu je, že nepotrebuje externý zdroj presného času. Cieľom je hlavne vzájomná synchronizácia uzlov.

5.17 Časové pečiatky

Časová pečiatka je hodnota priradená udalosti alebo transakcii, ktorá určuje jej poradie.

V databázach sa časové pečiatky používajú napríklad:

- na riadenie súbehu,
- na usporiadanie transakcií,
- na zisťovanie príčinného poradia udalostí,
- pri replikačných protokoloch.

Ak máme transakcie:

$$T_1, T_2,$$

a platí:

$$TS(T_1) < TS(T_2),$$

potom systém môže vyžadovať, aby sa konfliktné operácie správali tak, ako keby T_1 prebehla pred T_2 .

5.18 Lamportov protokol logických hodín

Fyzické hodiny nie sú vždy potrebné. V distribuovaných systémoch často stačí zachytiť kauzálne poradie udalostí.

Lamport zaviedol reláciu:

$$\rightarrow$$

nazývanú **happened-before**.

Platí:

1. Ak udalosti a a b nastali v tom istom procese a a nastala pred b , potom:

$$a \rightarrow b.$$

2. Ak a je odoslanie správy a b je prijatie tej istej správy, potom:

$$a \rightarrow b.$$

3. Relácia je tranzitívna:

$$a \rightarrow b \wedge b \rightarrow c \Rightarrow a \rightarrow c.$$

Každý proces P_i má logické hodiny:

$$LC_i.$$

Pravidlá Lamportových hodín:

1. Pred každou lokálnou udalosťou proces zvýši svoje hodiny:

$$LC_i := LC_i + 1.$$

2. Pri odoslaní správy priloží hodnotu:

$$LC_i.$$

3. Pri prijatí správy s časom t nastaví:

$$LC_i := \max(LC_i, t) + 1.$$

Lamportove hodiny zaručujú:

$$a \rightarrow b \Rightarrow LC(a) < LC(b).$$

Opačný smer však neplatí všeobecne. Z toho, že:

$$LC(a) < LC(b),$$

nemusí vyplývať:

$$a \rightarrow b.$$

Na úplné usporiadanie udalostí možno použiť dvojicu:

$$(LC(e), ID_{procesu}).$$

5.19 Krátke zhrnutie na skúšku

Distribúovaná databáza pozostáva z viacerých uzlov, pričom každý uzol má lokálny databázový systém a správcu uzla. Systém má zachovať ACID vlastnosti, transparentnosť a odolnosť voči výpadkom. Najdôležitejším problémom je atomický commit: všetky uzly musia rozhodnúť rovnako, teda buď všetky COMMIT, alebo všetky ABORT.

Dvojfázový commit má fázu hlasovania a fázu rozhodnutia. Ak všetci účastníci hlasujú **YES**, koordinátor rozhodne **COMMIT**; ak aspoň jeden hlasuje **NO**, rozhodne **ABORT**. Nevýhodou 2ACP je blokovanie pri výpadku koordinátora. Trojfázový commit pridáva fázu **PRECOMMIT**, ktorá umožňuje účastníkom po výpadku koordinátora bezpečnejšie zvoliť nové rozhodnutie. Pri sieťových partíciách však môže blokovat každý ACP.

Ak koordinátor zlyhá, volí sa nový. Bully algoritmus vyberie nezlyhaný uzol s najväčším identifikátorom. Pri replikácii je jeden objekt uložený na viacerých uzloch, čo zvyšuje dostupnosť, ale komplikuje izoláciu. Tú možno riešiť distribuovaným zamykaním, primary-copy schémou alebo quorum protokolom. Quorum protokol vyžaduje, aby sa čítacie a zápisové quorá pretínali.

Distribuované deadlocky sa zisťujú pomocou lokálnych wait-for grafov, z ktorých koordinátor skladá globálny wait-for graph. Cyklus znamená deadlock, ale kvôli oneskoreniam správ môže dôjsť aj k falošnej detekcii.

Fyzické hodiny možno synchronizovať Christianovým protokolom alebo Berkeleyho protokolom. Christianov protokol používa časový server a odhad oneskorenia správy. Berkeleyho protokol synchronizuje skupinu uzlov podľa priemernej korekcie. Lamportove logické hodiny neriešia reálny čas, ale kauzálne poradie udalostí. Zaručujú, že ak udalosť a kauzálne predchádza udalosti b , potom časová pečiatka a je menšia než časová pečiatka b .